Linux Systems and Performance

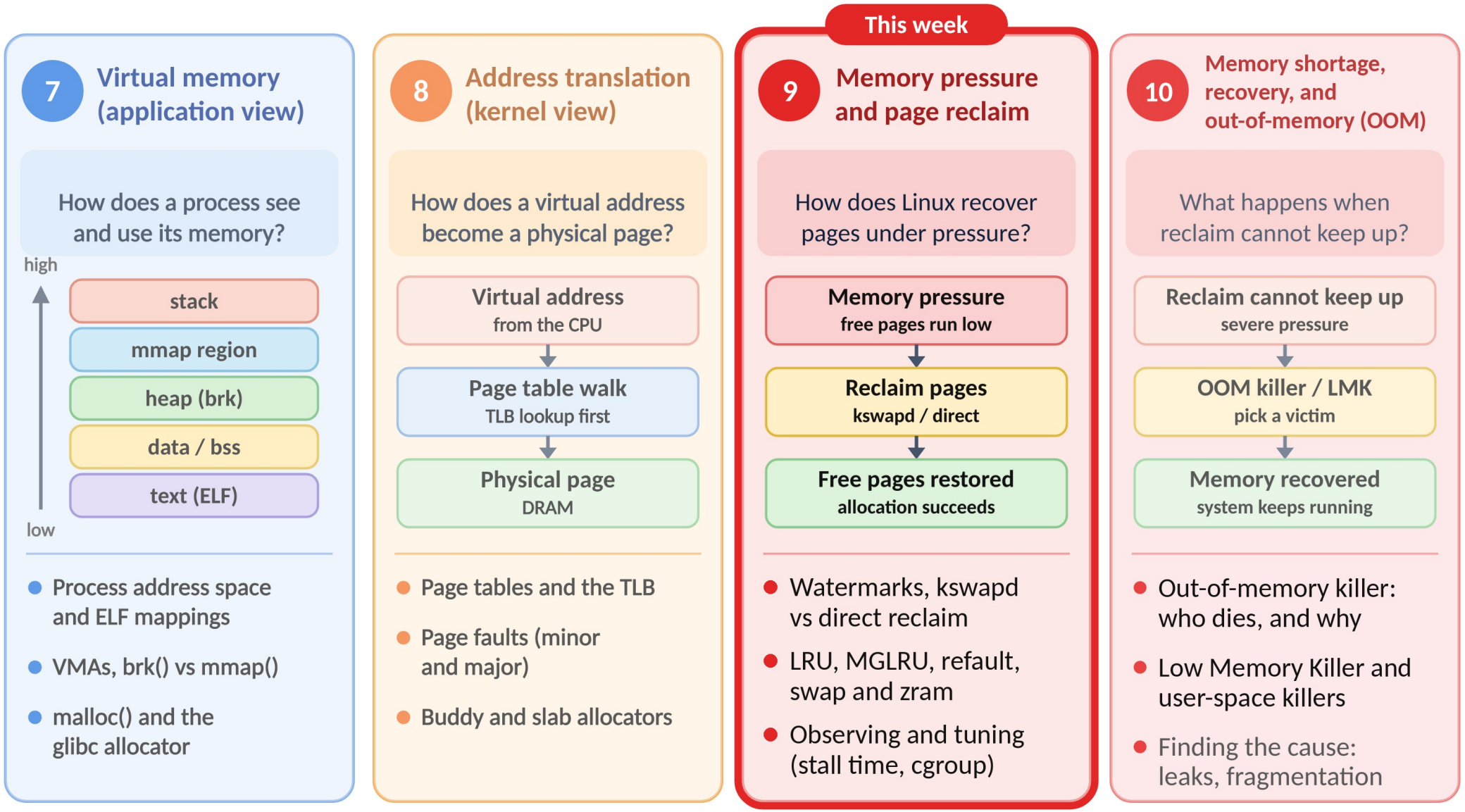
Part 3: Memory Management

Week 9: Memory Pressure and Page Reclaim

Sungyong Park, Ph.D.
Data-Intensive AI Computing and Systems (DISCOS) Laboratory
Department of Computer Science and Engineering, Sogang University
URL: https://discos.sogang.ac.kr
E-mail: parksy@sogang.ac.kr



Roadmap of Part 3



Where Does Week 9 Fit in the Page Allocation Path?

Key Points

- **Recap: three stages**
  Page allocation has three stages: the **fast path**, the **slow path**, and **failure handling**.
- **Fast path**
  Week 8: a free page is taken right away when a zone has enough free pages.
- **Slow path**
  When allocation is hard, Linux **reclaims** used memory to secure **free memory**, then allocates.
- **Takes more time**
  Reclaim needs work and waiting, so the slow path is **slower** than the fast path.
- **This week's focus**
  Week 9 covers **how reclaim secures free memory** on the slow path.

Where Does Week 9 Fit in the Page Allocation Path?

Reclaim secures free memory on the slow path.

Allocation Path: Three Stages

Week 8
Fast path
a free page is available now

→

This week: Week 9
Slow path
reclaim, then allocate

→

Week 10
Failure
OOM killer, Low Memory Killer

▼ fast path fails

Slow Path in Three Steps (Simplified)

1 Allocation is hard
The fast path cannot find enough free memory right away.

→

2 Reclaim memory
Linux frees pages that are in use, to secure enough **free memory**.

→

3 Allocate
The request is served from the new free memory. This takes more time than the fast path.

Where this week goes deeper

Who reclaims and when → what is reclaimed first (LRU lists, swap, kernel memory) → how to measure, control, and diagnose it (PSI, cgroups, webOS TV).

Simplified path. The real path varies with GFP flags, allocation order, and reclaim state.
Kernel behavior follows mainline Linux 6.x; device kernels may differ.

Key Takeaway

Week 8 covered the fast path; Week 9 covers how reclaim secures free memory on the slow path; Week 10 covers failure handling.

PART 3: Memory Management

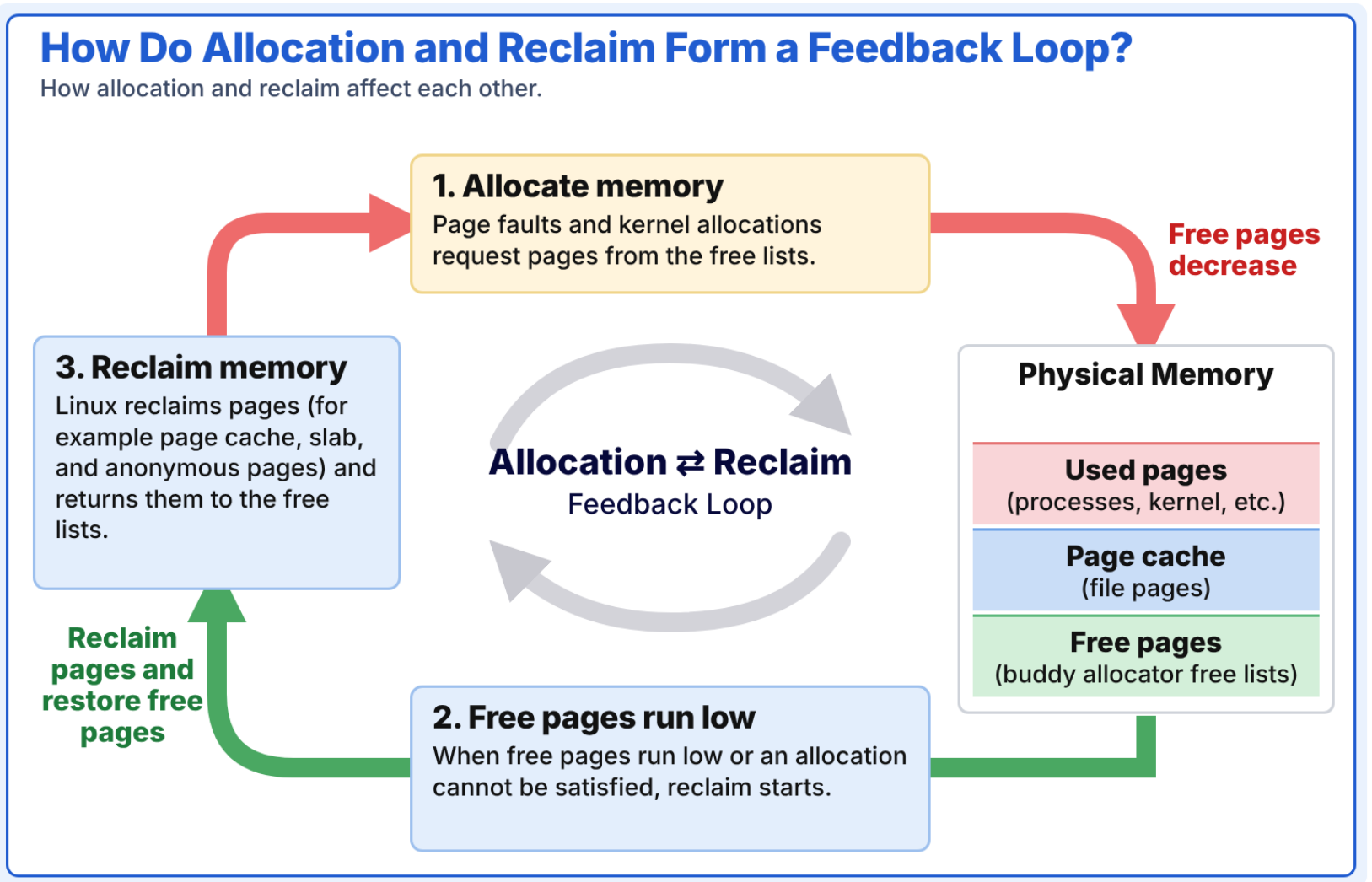
- **Week 9**

- **Memory Pressure and Reclaim Fundamentals**
  - When does Linux decide that memory must be reclaimed?
- **Page Reclaim Policy, LRU, and Swap**
  - Which pages does Linux reclaim, and how does it choose them?
- **Observability, Memory Cgroup, and webOS**
  - How do we observe and control reclaim, and tune it for webOS?

How Do Allocation and Reclaim Form a Feedback Loop?

Key Points

- **The loop in brief**
  Allocation removes **free pages**; reclaim puts them back, so each reacts to the other.
- **Allocation takes pages**
  Page faults and kernel allocations take pages from the **buddy allocator**.
- **Reclaim returns pages**
  Low free pages make Linux reclaim **page cache**, **slab**, and **anonymous** pages.
- **Low free by design**
  Reclaim keeps refilling, so **free memory** stays low even when the system is healthy.



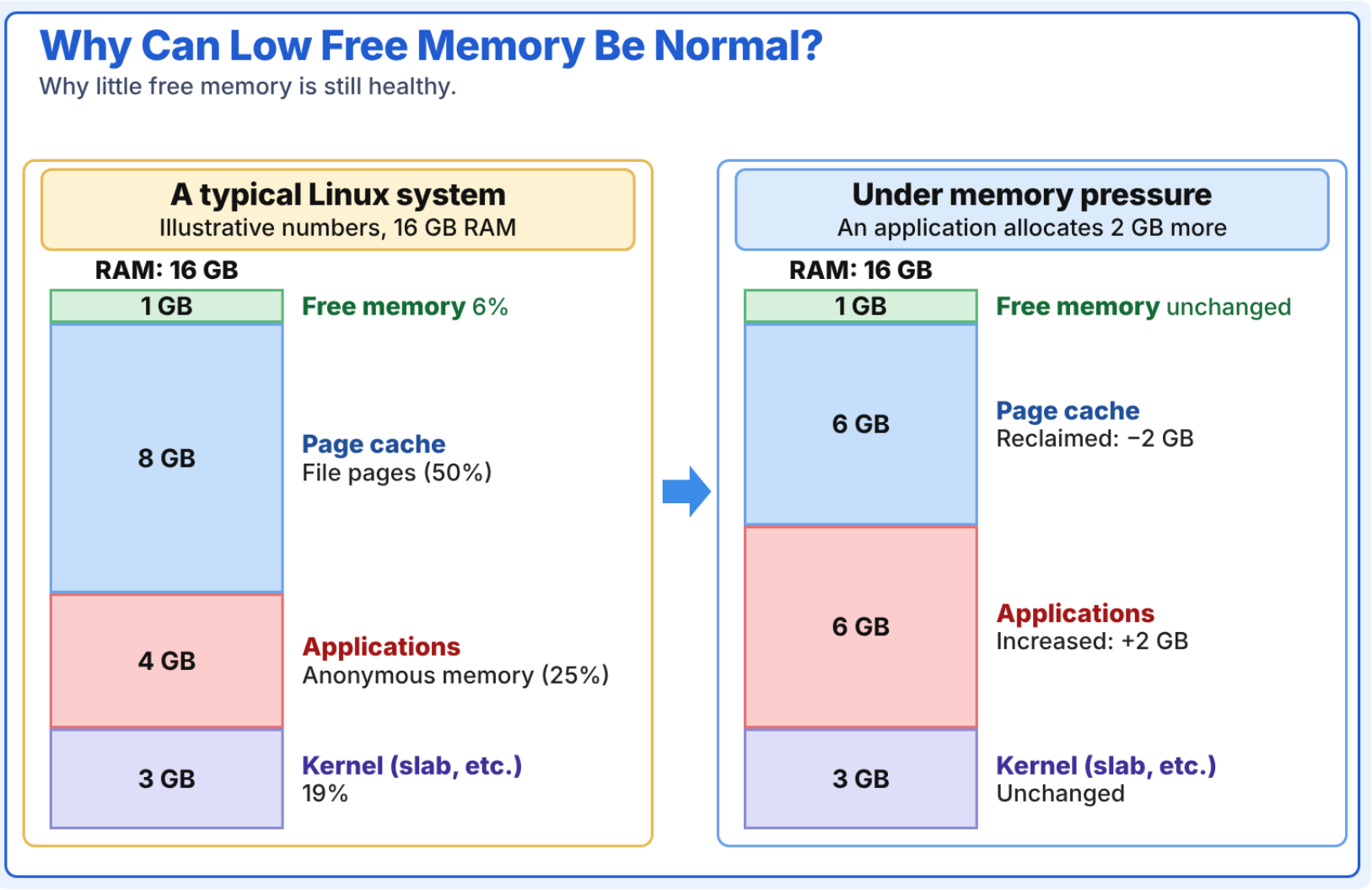
Key Takeaway

Allocation and reclaim form a feedback loop that keeps free memory low by design, so free memory alone cannot show whether memory is short.

Why Can Low Free Memory Be Normal?

Key Points

- **Low free is healthy**
  Linux uses otherwise unused RAM as **page cache**, so free memory stays small.
- **Used is not unavailable**
  Memory shown as **used** often holds cache that Linux can release.
- **A 16 GB example**
  Illustratively, 1 GB is free, but 8 GB of **page cache** is mostly releasable.
- **Cache helps speed**
  Cached files reduce **disk reads**; Linux reclaims the cache when needed.



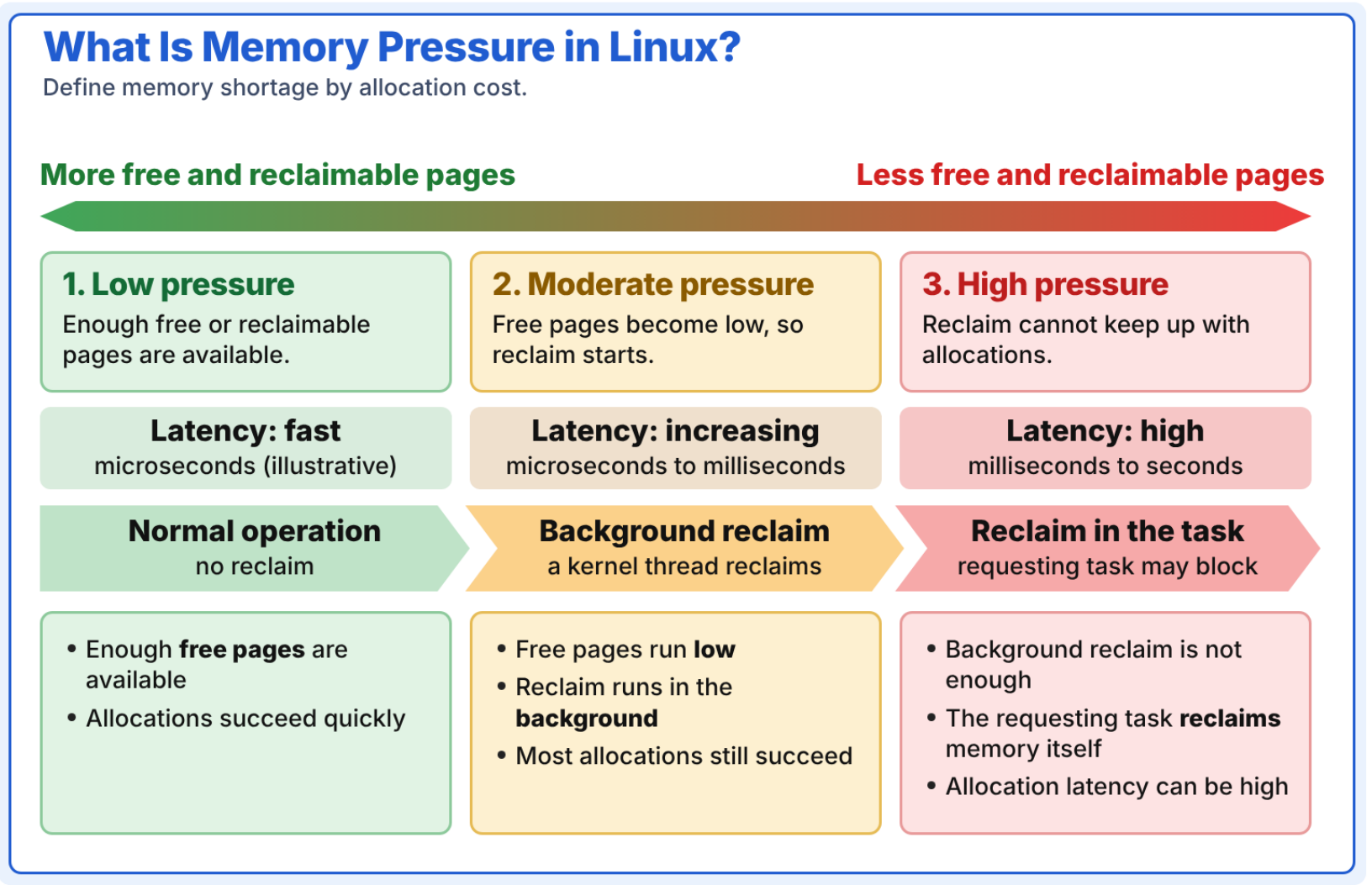
Key Takeaway

Low free memory is normal because Linux uses unused RAM as page cache and reclaims it automatically when new allocations need memory.

What Is Memory Pressure in Linux?

Key Points

- **Shortage is cost**
  Memory is short when a new **allocation** needs more reclaim work or waiting, not when free memory is low.
- **A gradual range**
  It rises **gradually** as free and **reclaimable** pages shrink; it is not on or off.
- **How Linux responds**
  Linux first reclaims in the **background**; if that is not enough, the requesting task **reclaims itself**.
- **What tasks see**
  Pressure causes longer **allocation latency** and stalls in the requesting task.



Key Takeaway

Memory pressure, the rising cost of satisfying new allocations, defines memory shortage; low free memory alone does not.

Which Zone Serves an Allocation Request?

Key Points

- **Zones split memory**
  Each node's memory is divided into **zones** by address: **DMA**, **DMA32**, and **Normal**.
- **Requests set a limit**
  Each request names the **highest zone** it may use.
- **Order goes downward**
  An allocation tries that zone first, then **lower zones**, never upward.
- **Failure means slow path**
  A zone with free pages above **low** serves the request (**fast path**). If none does, it enters the **slow path**.
- **Per-zone accounting**
  Free pages and **watermarks** are kept **per zone**.

Which Zone Serves an Allocation Request?

Which zones a request may use.

Order tried → Zone for each request	Normal highest addresses	DMA32 below 4 GiB	DMA below 16 MiB
Request that allows Normal `GFP_KERNEL`	1 1st choice	2 2nd choice	3 3rd choice
Request that allows up to DMA32 `GFP_DMA32`	not allowed	1 1st choice	2 2nd choice
Request that needs DMA `GFP_DMA`	not allowed	not allowed	1 1st choice

Example: x86-64 with one memory node. Zone sets and sizes vary by architecture, and each memory node has its own zones. A request that allows a higher zone may use lower zones, but never the reverse.

Key Takeaway

An allocation tries the highest zone it may use and falls back to lower zones, never upward.

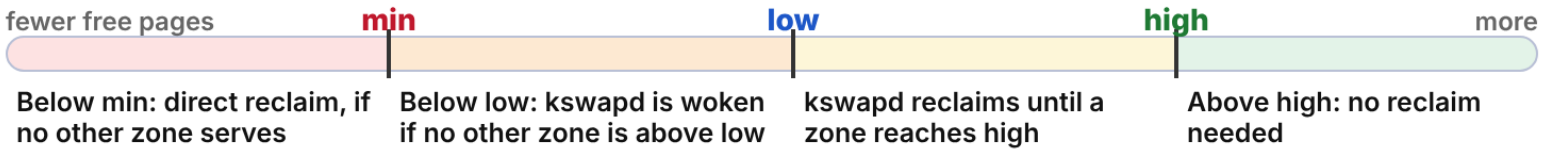
How Does Linux Reclaim Free Memory?

Key Points

- **Reclaim per zone**
  Reclaim frees pages **zone by zone**, using each zone's **watermarks**.
- **Slow path starts reclaim**
  Reclaim usually starts in the **slow path**: with no zone above **low**, **kswapd** starts; at **high**, it stops.
- **kswapd can fall behind**
  kswapd is one thread per node, so **bursts** can outpace it and lead to direct reclaim.
- **Direct reclaim**
  When every usable zone is below **min**, the task runs **direct reclaim** and **waits**.
- **Atomic requests**
  Atomic requests cannot wait, so they use the reserve below **min**.

How Does Linux Reclaim Free Memory?

Who frees memory, and when.



Below min	Below low	low → high	Above high
direct reclaim, if no other zone serves	kswapd is woken if no other zone is above low	kswapd reclaims until a zone reaches high	no reclaim needed

kswapd

Background reclaim

Runs in	A kernel thread per **node**, separate from the allocating task
Starts when	The **slow path** is entered: no usable zone is above **low**
Stops when	Its node is balanced: an eligible zone reaches **high** (plus any boost)
Effect	The allocating task usually **does not wait**
Limit	One thread per node: **bursts**, large allocations, or slow write-back can outpace it

Direct reclaim

Reclaim inside the allocation

Runs in	The **allocating task** itself
Starts when	Every usable zone is below **min**, and the request can wait
Stops when	Enough pages are freed for this request
Effect	The task **waits**, so allocation latency rises
Cause	**kswapd fell behind** the allocation rate

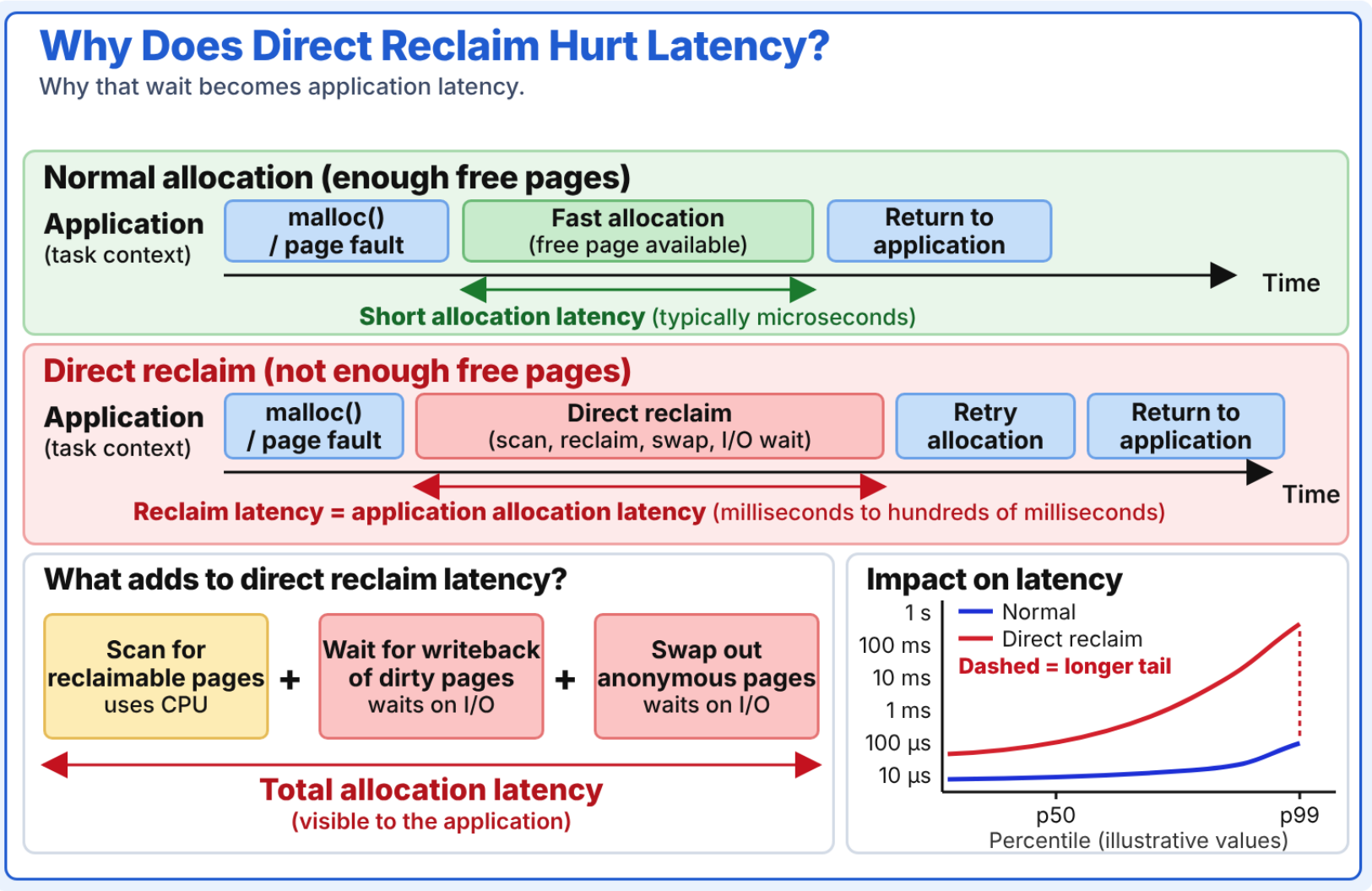
Key Takeaway

Reclaim works zone by zone against watermarks: kswapd frees pages in the background, and direct reclaim frees them while the task waits.

Why Does Direct Reclaim Hurt Latency?

Key Points

- **Task is blocked**
  Direct reclaim runs in the **requesting task**, which cannot continue until it ends.
- **Reclaim time is latency**
  All reclaim time adds to the **allocation latency** the application sees.
- **Several steps add up**
  Scanning, **swap-out**, and **I/O waits** each lengthen the delay.
- **Slow cases hurt the tail**
  With swap or slow storage it can take milliseconds to hundreds, raising **tail latency**.



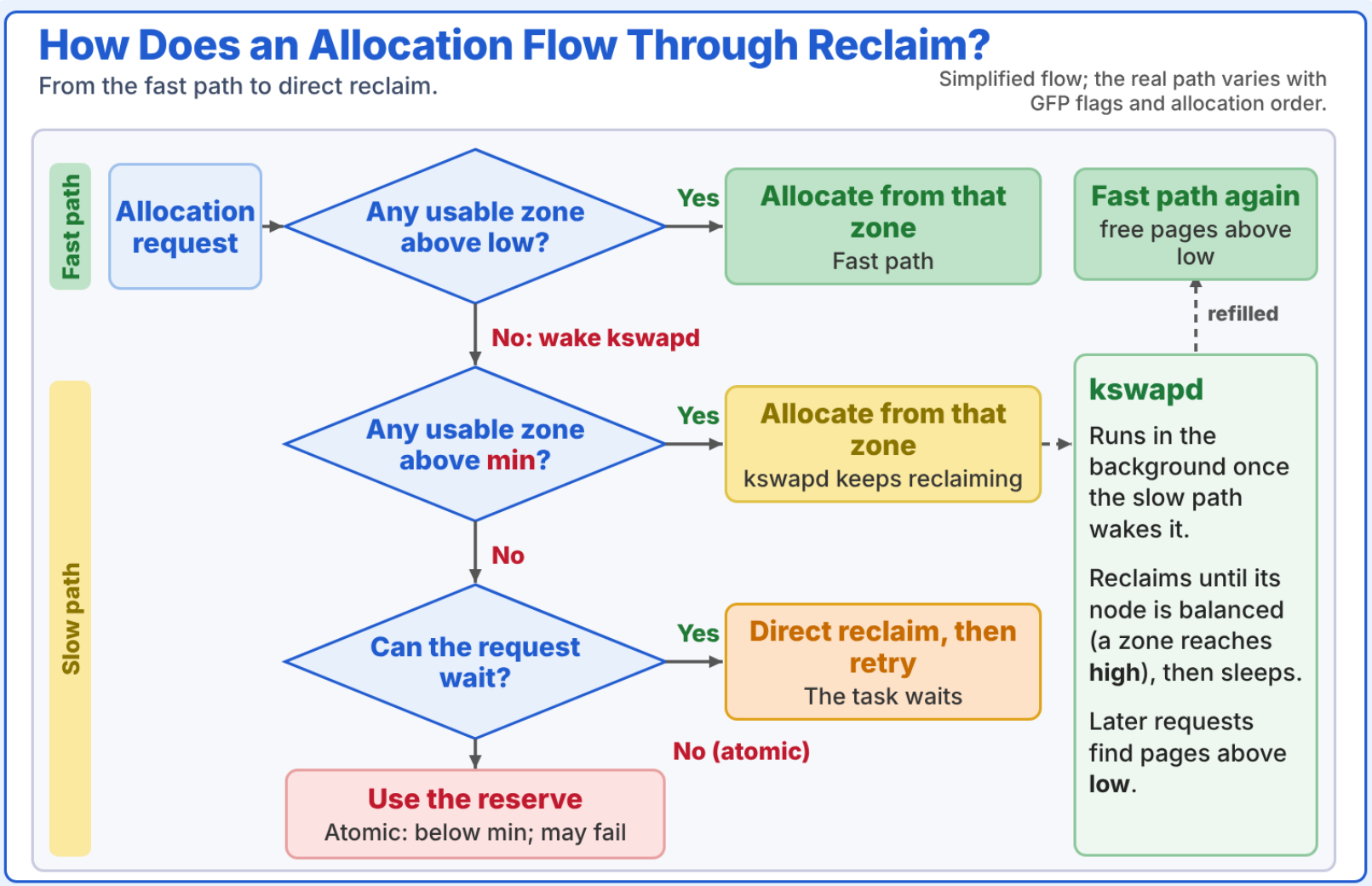
Key Takeaway

Direct reclaim makes the requesting task wait, so reclaim time becomes allocation latency and raises tail latency under memory pressure.

How Does an Allocation Flow Through Reclaim?

Key Points

- **Fast path first**
  If any usable zone is above **low**, the allocation succeeds at once.
- **Slow path: kswapd**
  Otherwise **kswapd** is woken and the zones are checked against **min**.
- **Above min still works**
  Any zone above **min** serves the request while **kswapd** keeps reclaiming.
- **Direct reclaim last**
  If every usable zone is below **min**, the task runs **direct reclaim**.
- **Atomic requests**
  Atomic requests cannot wait, so they use the reserve and may fail.



Key Takeaway

An allocation succeeds above low, still succeeds above min while kswapd reclaims, and reaches direct reclaim when every usable zone is below min.

Why Does Linux Use Watermarks?

Key Points

- **Reclaim takes time**
  Scanning, **writeback**, and **swap-out** take time, so reclaim cannot wait until free pages reach zero.
- **A reserve buys time**
  Watermarks keep a free-page **reserve** while reclaim is running.
- **Bursts**
  A **burst** of allocations can use free pages faster than reclaim frees them.
- **Atomic allocations**
  Atomic allocations cannot sleep to wait for reclaim, so they use the reserve.
- **Reclaim needs pages**
  Reclaim itself, such as **writeback** and **swap-out**, needs free pages to make progress.

Why Does Linux Use Watermarks?

Why a reserve of free pages is kept.

Reclaim only when free pages reach zero

Free pages reach zero → The next allocation cannot be served → Reclaim starts now, and the task waits → Atomic requests may fail

Reclaim starts when no usable zone is above low

No zone is above low, but pages remain → kswapd starts in the background → Allocations are still served → Free pages recover before the reserve runs out

Why must the reserve exist?

Bursts
Allocations can arrive faster than reclaim frees pages.
Without a reserve: free pages reach zero first.

Atomic allocations
Interrupt handlers cannot sleep to wait for reclaim.
Without a reserve: they may fail.

Reclaim itself
Writeback and swap-out need memory to run.
Without a reserve: reclaim cannot make progress.

Reclaim takes time (scanning, writeback, swap-out), so it must start while free pages remain.

Key Takeaway

Watermarks keep a free-page reserve so reclaim can start early: reclaim takes time, and some allocations cannot wait.

How Are the Watermark Levels Determined?

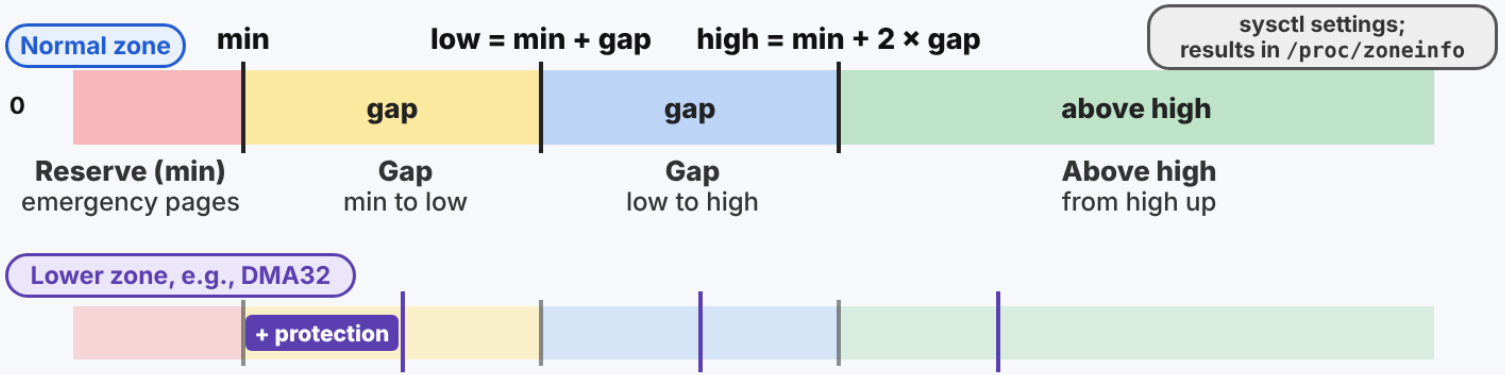
Key Points

- **min sets the reserve**
  min comes from the **vm.min_free_kbytes** sysctl, shared among zones by size.
- **gap sets the spacing**
  The **gap** comes from **vm.watermark_scale_factor**; **low** = min + gap and **high** = min + 2 × gap.
- **Zone protection**
  vm.lowmem_reserve_ratio raises a lower zone's watermark for higher-zone requests.
- **Why these defaults**
  min serves requests that cannot wait; the **gap** gives **kswapd** time to catch up.
- **Cost of a larger gap**
  kswapd starts earlier and runs longer, so less memory stays in use as cache.

How Are the Watermark Levels Determined?

Where the levels sit, how they are set, and why.

Where the levels sit in a zone's free pages (illustrative, not to scale)



lowmem protection: a lower zone (DMA, DMA32) adds this value to its watermark when a request that prefers a higher zone falls back to it.

How the defaults are computed, and why

- **Why a reserve:** Atomic requests and reclaim itself cannot wait, so some pages must stay free.
- **Why a gap:** kswapd needs time to free pages while allocations keep using them.
- **Why high = 2 × gap:** kswapd frees a batch, then sleeps instead of waking again at once.
- **Why the gap scales with zone size:** Larger zones see larger allocation bursts.
- **Watermark boost:** Fragmentation can temporarily raise the levels.

Key Takeaway

min comes from min_free_kbytes and the gap from watermark_scale_factor; low and high are derived from them.

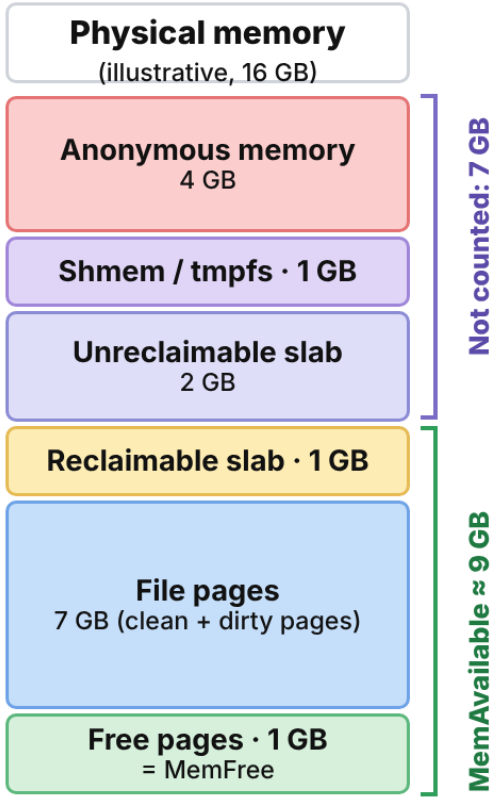
What Is the Difference Between MemFree and MemAvailable?

Key Points

- **Two measures**
  MemFree counts unused pages; **MemAvailable** counts memory reclaimable **without swap**.
- **What it adds**
  It adds **file pages** (dirty pages after writeback) and reclaimable **slab**; no swap needed.
- **A 16 GB example**
  Illustratively, **/proc/meminfo** shows MemFree of 1 GB and MemAvailable of about 9 GB.
- **Why it excludes some**
  Anonymous memory and **Shmem** need **swap** to be reclaimed, so they are excluded.
- **Each has a use**
  MemFree sums zone free pages; **MemAvailable** estimates headroom. Watermarks use neither.

What Is the Difference Between MemFree and MemAvailable?

What each field tells us.



MemFree ≈ 1 GB

- **Free-list** pages, summed over zones.

MemAvailable ≈ 9 GB

- Obtainable without swapping: free + part of cache and slab.

```
$ grep -E 'MemFree|MemAvailable' /proc/meminfo
MemFree:         1048576 kB
MemAvailable:    9437184 kB
```

Which pages does MemAvailable count?

Page type	How it is reclaimed	Needs swap?	Counted?
File pages, clean	Dropped; re-read if needed	No	**Counted**
File pages, dirty	Written to its file, then dropped	No	**Counted**
Reclaimable slab	Freed when unused	No	**Counted**
Anonymous memory	No backing file: to swap	Yes	**Not counted**
Shmem / tmpfs	No backing file: to swap	Yes	**Not counted**
Unreclaimable slab	In use; cannot be freed	n/a	**Not counted**

Illustrative 16 GB example; the kernel counts only part of cache and slab.

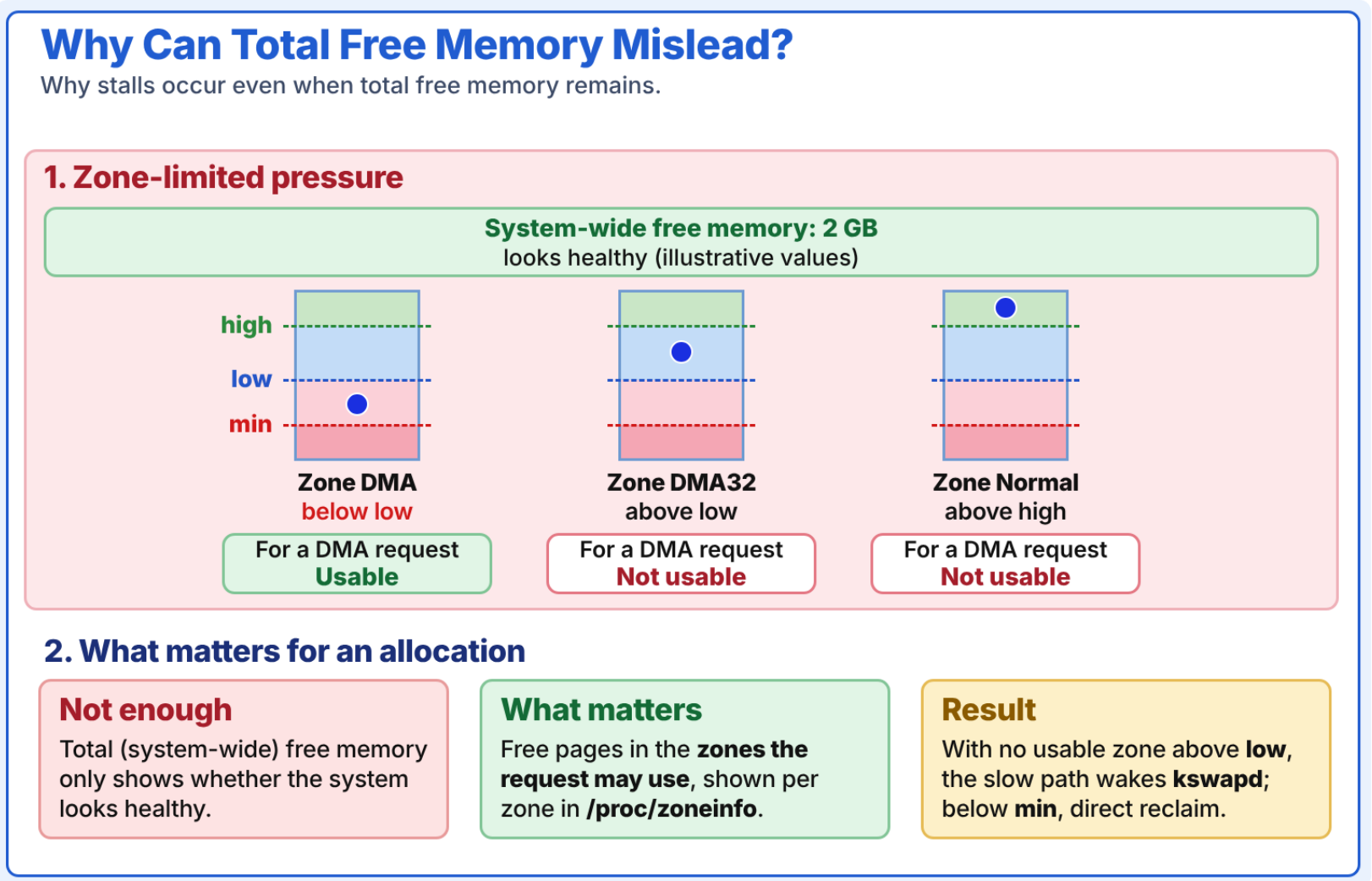
Key Takeaway

MemAvailable counts only memory reclaimable without swap; anonymous memory and Shmem need swap to be reclaimed, so they are excluded.

Why Can Total Free Memory Mislead?

Key Points

- **Total free can mislead**
  System-wide free pages do not guarantee a **particular allocation** succeeds.
- **Requests have limits**
  Each request may use only some **zones**, depending on its memory type.
- **Fallback has limits**
  Fallback goes **downward** only, so a low-zone request cannot use free pages above it.
- **Reclaim follows**
  When no usable zone has enough free pages, the **slow path** starts reclaim.



Key Takeaway

Total free memory can mislead: when the zones a request may use lack free pages, Linux reclaims even though total free memory remains.

Checkpoint: Is It Background or Direct Reclaim?

Key Points

- **Two reclaim paths**
  Is it **kswapd** in the background, or the task itself reclaiming **directly**?
- **Slow path: kswapd**
  With no usable zone above **low**, the slow path wakes **kswapd**.
- **Below min: direct**
  With every usable zone below **min**, the task runs **direct reclaim** and waits.
- **Atomic cannot wait**
  Atomic requests cannot sleep, so they use reserves or fail.
- **Confirm with counters**
  Rising **pgscan_direct** or **allocstall** means direct reclaim.

Checkpoint: Is It Background or Direct Reclaim?

Classify situations by path.

#	Situation	Which reclaim path?	Reason
1	No usable zone is above low, but some usable zone is above min.	Background reclaim (kswapd)	The slow path wakes kswapd; the allocation can still succeed above min.
2	A task that can wait allocates, and every usable zone is below min.	Direct reclaim (requesting task)	The task reclaims in its own context and can stall.
3	kswapd is running, but free pages keep falling and allocstall_normal rises.	Both: kswapd and direct reclaim	kswapd cannot keep up, so tasks reclaim directly.
4	MemFree is low, but MemAvailable is high.	No immediate pressure	Much memory is reclaimable; low MemFree alone is not pressure.
5	An interrupt handler allocates with GFP_ATOMIC while every usable zone is below min.	No direct reclaim (cannot sleep)	Atomic context cannot sleep: it uses reserves or fails; kswapd is woken.

GFP = Get Free Pages; kswapd = background reclaim thread.

Key Takeaway

Identify the trigger and context: no usable zone above low → kswapd; every usable zone below min → direct reclaim; low MemFree alone is not pressure.

PART 3: Memory Management

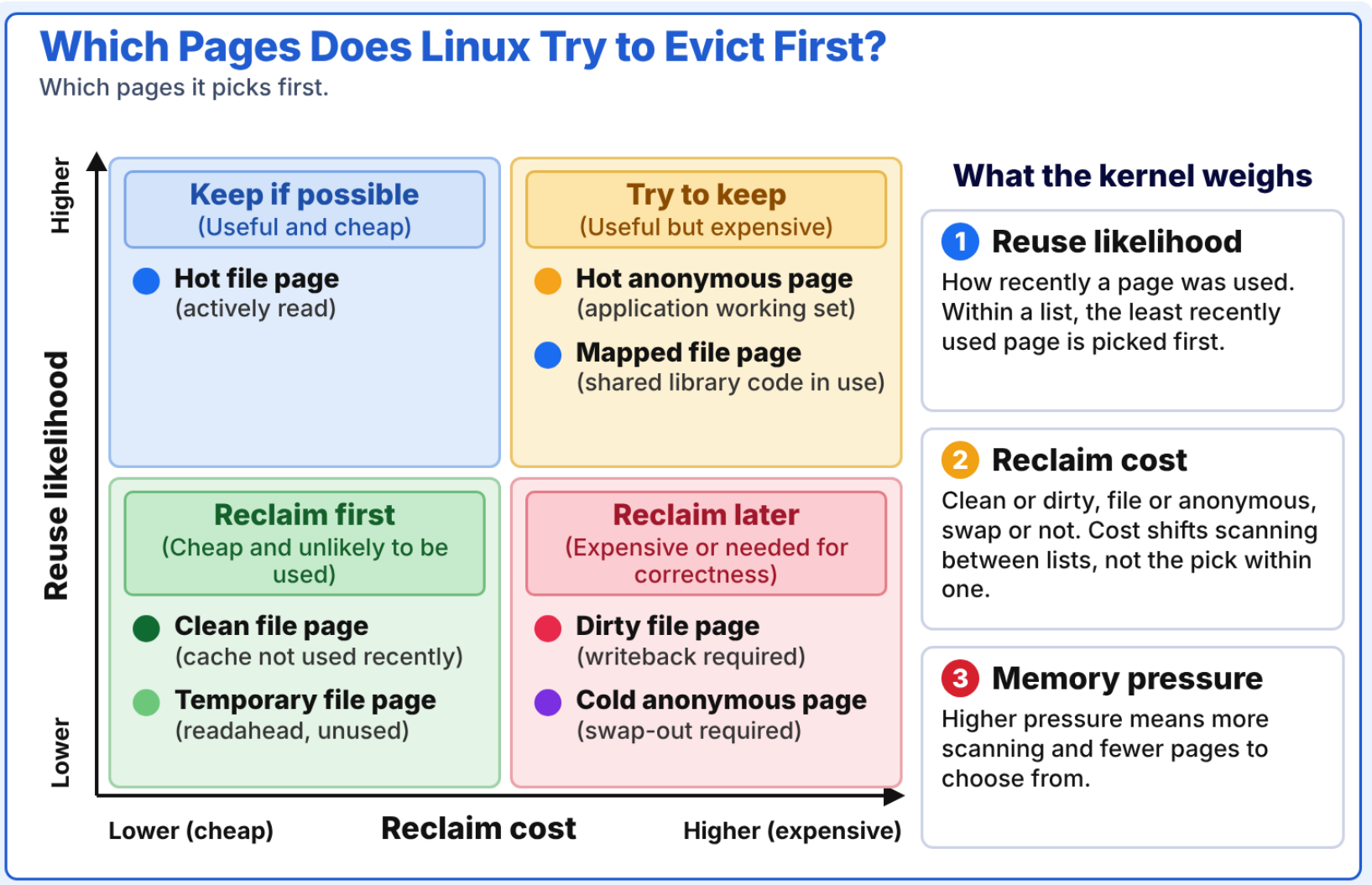
- **Week 9**

- Memory Pressure and Reclaim Fundamentals
  - When does Linux decide that memory must be reclaimed?
- **Page Reclaim Policy, LRU, and Swap**
  - Which pages does Linux reclaim, and how does it choose them?
- Observability, Memory Cgroup, and webOS
  - How do we observe and control reclaim, and tune it for webOS?

Which Pages Does Linux Try to Evict First?

Key Points

- **Cheap, cold pages first**
  Linux evicts pages that are **cheap to reclaim** and **unlikely to be reused**.
- **Two factors decide**
  Recency picks pages within a list; **cost** shifts scanning between lists.
- **Page types differ**
  Clean file pages are cheapest; **dirty file** and **anonymous** pages cost more.
- **Pressure shifts the mix**
  Higher **memory pressure** means more scanning and fewer pages to choose from.
- **No fixed order**
  History, **state**, and pressure all count.



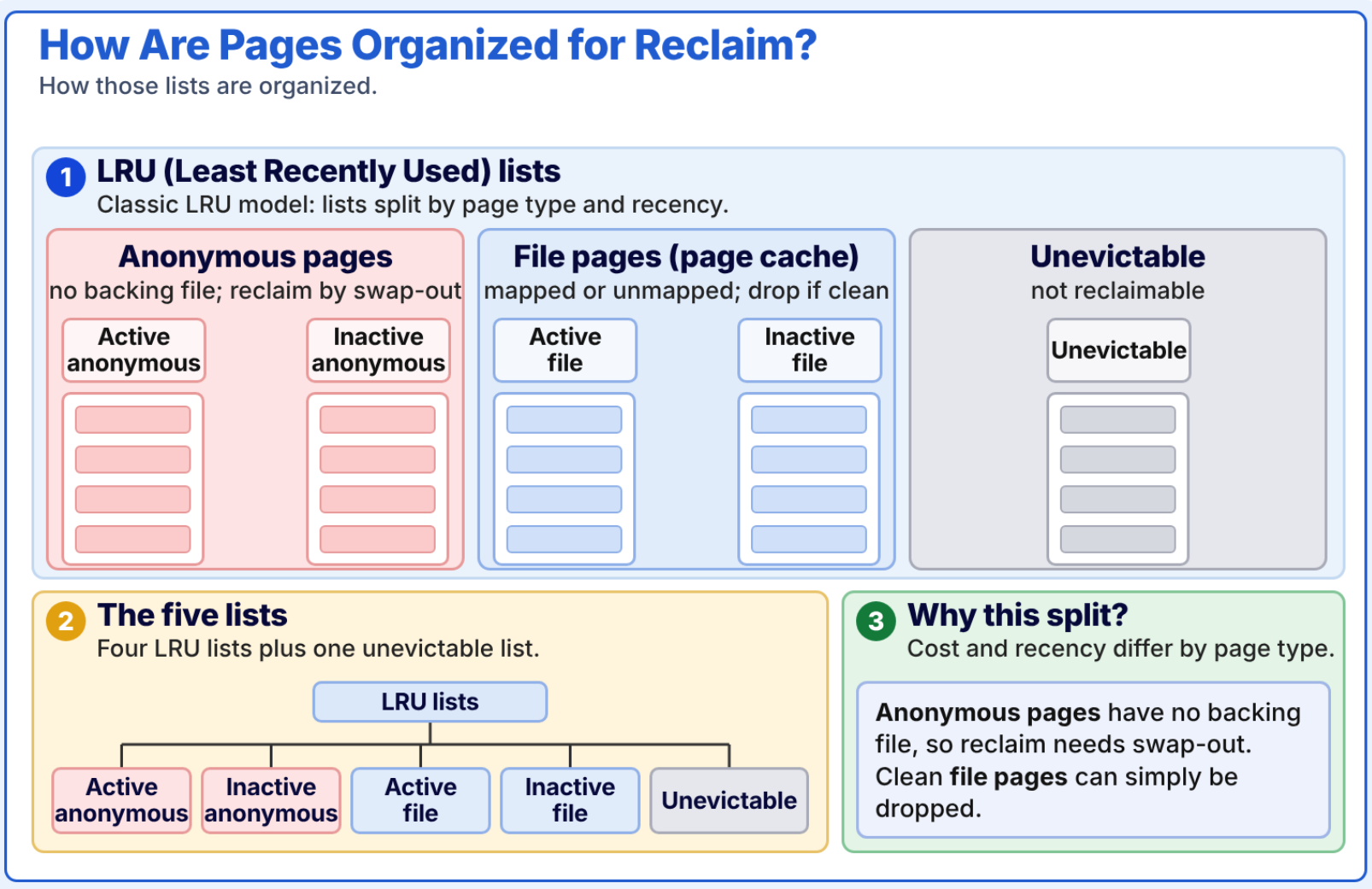
Key Takeaway

Linux evicts cold, cheap-to-reclaim pages first: recency picks pages within a list, cost shifts scanning between lists, and memory pressure sets how hard it scans.

How Are Pages Organized for Reclaim?

Key Points

- **Pages sit on LRU lists**
  Reclaim picks from **Least Recently Used** lists, not from all memory.
- **Four lists plus one**
  Anonymous pages have no backing file. **File** pages are page cache: mapped (mmap) or unmapped (read/write).
- **Active and inactive**
  Active lists hold recently used pages; **inactive** lists hold reclaim candidates.
- **Unevictable pages**
  mlock() pages stay on the **unevictable list** and are skipped.
- **Inactive tails first**
  Scans the tail of **inactive lists** first.



Key Takeaway

Linux keeps four LRU lists (active/inactive × anonymous/file) plus an unevictable list, and reclaim mainly scans the tail of the inactive lists.

How Does Linux Manage LRU Lists?

Key Points

- **Pages are list entries**
  Each page descriptor holds an **lru** link, so the page itself sits on an LRU list.
- **An lruvec holds lists**
  An **lruvec** holds a list head per list and one **lru_lock**.
- **Two-way circular list**
  Every list is linked in both directions through the **lru** fields.
- **Tail holds the oldest**
  New pages join at the head, so the **tail** holds the oldest pages.
- **Batches cut locking**
  Pages are added and removed in **batches** per lock hold.

How Does Linux Manage LRU Lists?

How the LRU lists are built.

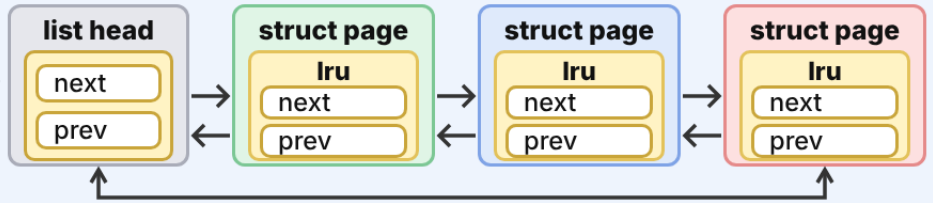
Classic LRU shown; MGLRU differs in details.

A Pages are entries of an LRU list

lruvec

- lists[0]: inactive anon
- lists[1]: active anon
- lists[2]: inactive file
- lists[3]: active file
- lists[4]: unevictable
- **lru_lock: one for all 5 lists**

Example: lists[2], a circular doubly linked list



Each pointer targets the **lru** field inside a page descriptor (struct page; struct folio in recent kernels). The kernel finds the page from that field.

Without memory cgroups: one lruvec per node. With cgroups: (# of cgroups × # of nodes) lruvecs.

B What the structure provides

1 List entry
Each page links itself into a list through its **lru** field, so no pages are copied.

2 List head
Each list has its own **list head** in the lruvec; new pages join at the head.

3 One lock
One **lru_lock** protects all five lists, so CPUs take turns. Handling pages in **batches** keeps lock overhead low.

Key Takeaway

Each lruvec holds five list heads and one lru_lock; pages join a list through their lru field, and the tail holds the oldest pages.

What Is a Page's Lifetime on the LRU Lists?

Key Points

- **New pages start inactive**
  A new page joins an **inactive** list as not referenced.
- **First access marks**
  A scan that finds the **accessed bit** set clears it and marks the page **referenced**.
- **Second access activates**
  A referenced page that is accessed again moves to the **active** list.
- **Reclaim deactivates**
  When an inactive list is small, reclaim moves active-tail pages to the **inactive** list.

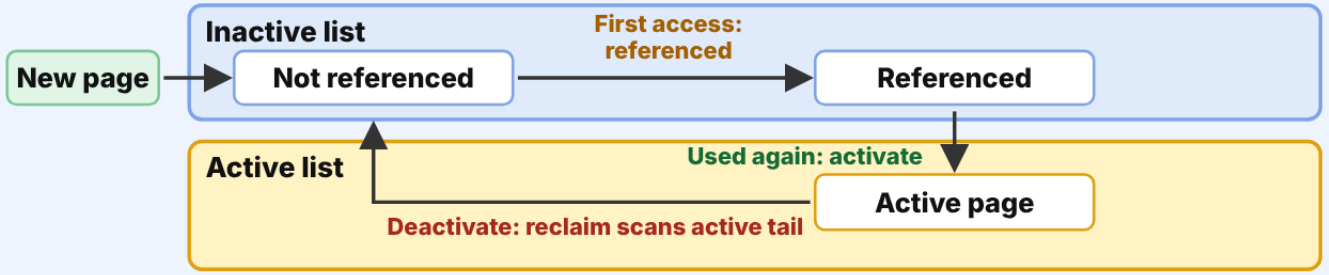
What Is a Page's Lifetime on the LRU Lists?

How one page moves between the two lists.

Classic LRU shown; MGLRU differs in details.

A Page lifetime: states and moves

Anon and file pages; unevictable pages never move.



B Step by Step (Mapped Pages)

Accessed bit: set by the CPU; unmapped file pages do not use it.

Step	Referenced flag	Accessed bit	Action	List after
Add	0	0	—	Inactive
First access	0	1	Clear bit, set flag	Inactive
Not accessed again	1	0	—	Inactive
Second access	1	1	Clear bit; move to active	Active
Deactivate	any	any	Clear bit; move to inactive	Inactive

Note: Unmapped file pages (read/write): a page accessed again while referenced is activated at once.

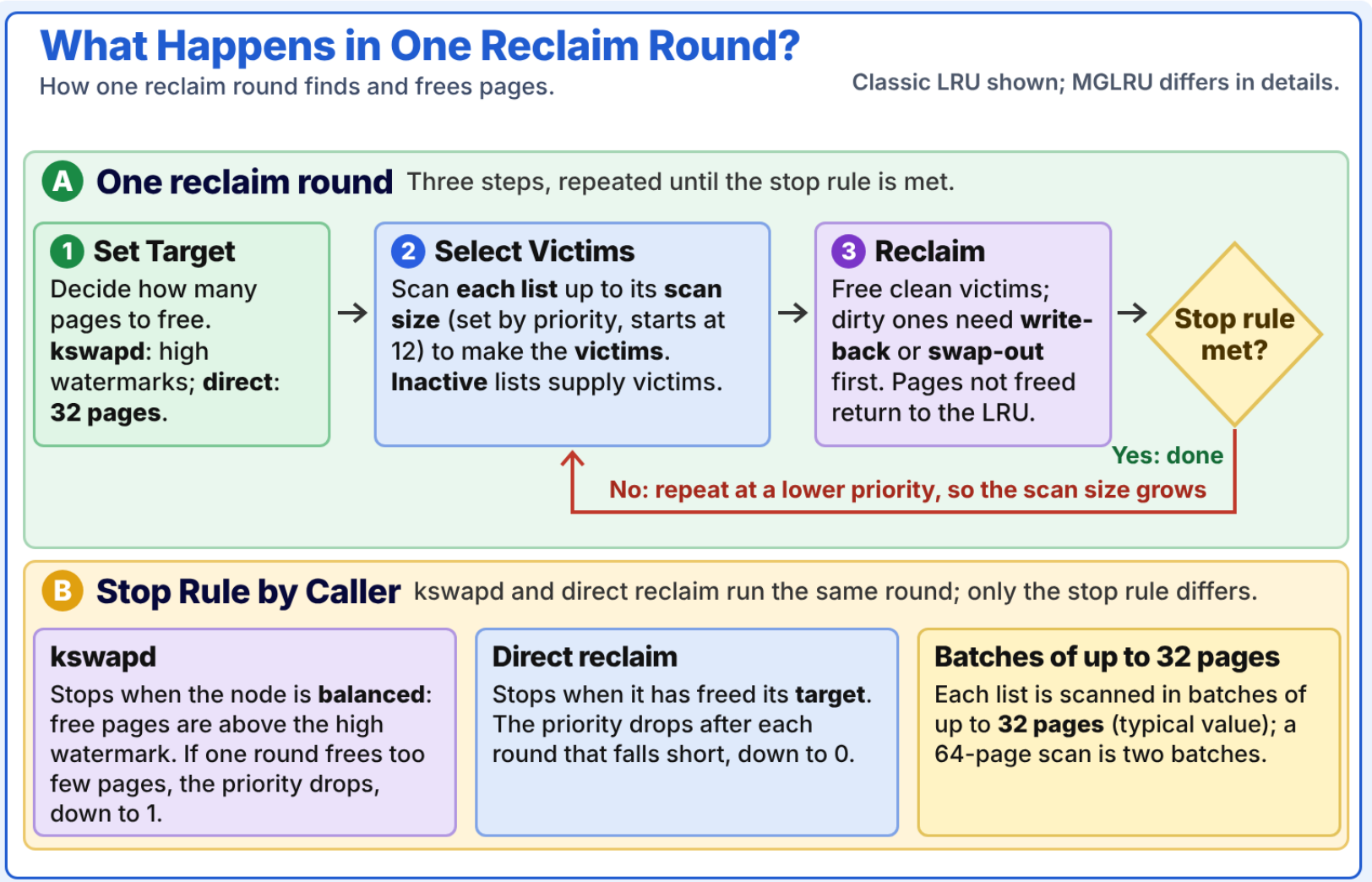
Key Takeaway

A page starts inactive, becomes active after a second access, and returns to inactive when reclaim deactivates it.

What Happens in One Reclaim Round?

Key Points

- **A round has three steps**
  Set a target, **select victims**, then **reclaim** them.
- **Each step has one job**
  Each step answers one question: how many, which pages, and what to do.
- **Callers differ**
  kswapd targets the high watermarks; **direct reclaim** targets 32 pages.
- **A round can repeat**
  If the stop rule is not met, a lower **priority** makes the next round scan a larger share.
- **Work runs in batches**
  Pages are processed in batches of up to **32 pages** (typical value).



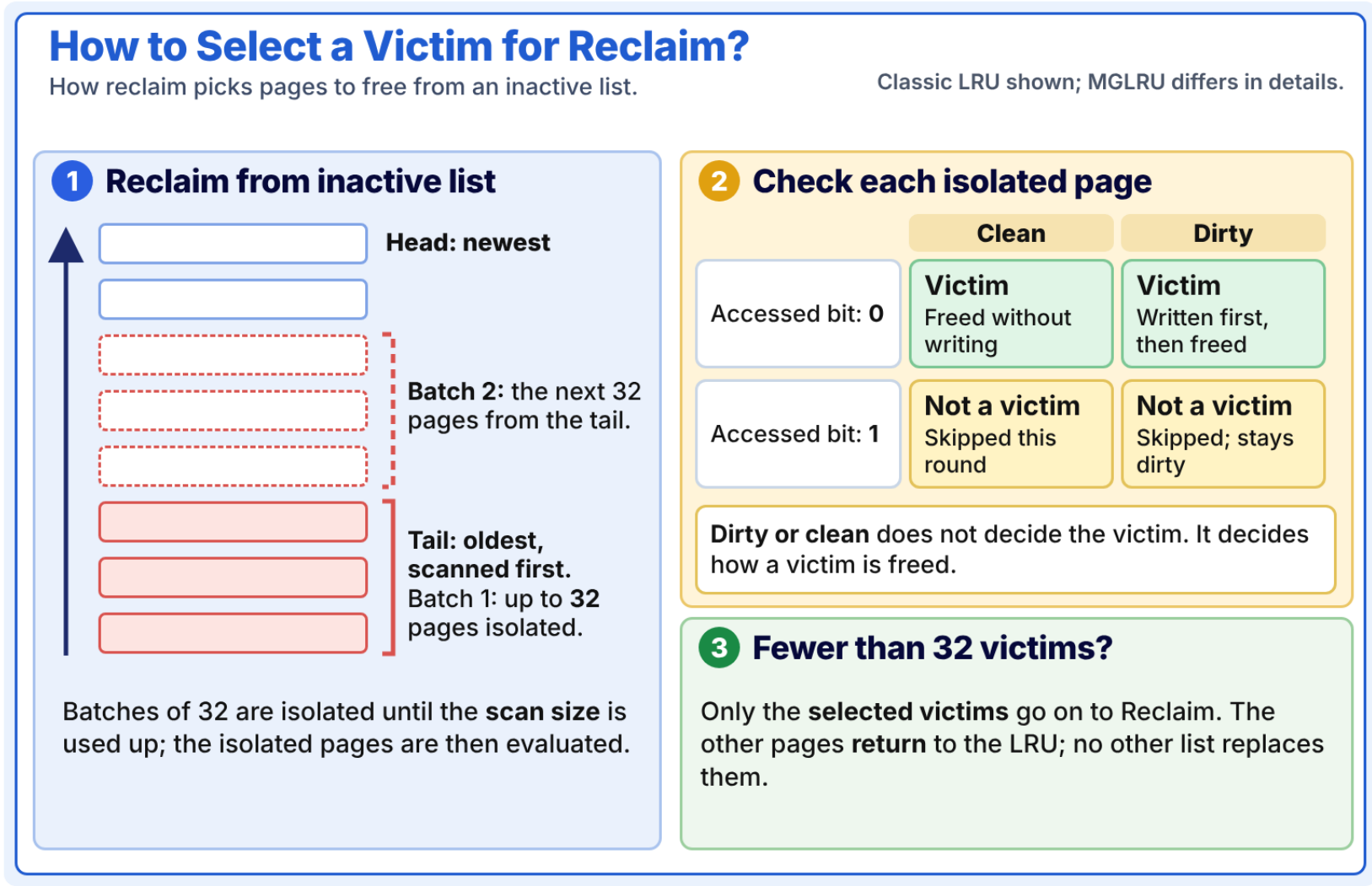
Key Takeaway

One reclaim round sets a target, selects victims, and reclaims them; if the stop rule is not met, the round repeats at a lower priority, which scans a larger share.

How to Select a Victim for Reclaim?

Key Points

- **Inactive list first**
  Victims come from **inactive** lists; the active list is scanned only to refill them.
- **Oldest pages first**
  Tail pages are the **least recently used**, so they are scanned first.
- **Check each page**
  A page whose **accessed bit** is not set becomes a victim; the others are not.
- **Short batches stay short**
  Pages that are not victims **return** to the LRU and are not replaced.
- **Scan size ends the scan**
  When the **scan size** is used up, the round repeats at a lower priority.



Key Takeaway

Reclaim scans batch by batch from the inactive tail: pages without the accessed bit become victims, the others return to the LRU, and the scan stops at its scan size.

What Happens to Each Victim?

Key Points

- **Clean pages drop**
  A **clean** page has a copy elsewhere, so it is dropped with no I/O.
- **Dirty pages need a write**
  A **dirty** page must be written out before it can be freed.
- **Flusher writes file pages**
  Reclaim sets the reclaim flag on a dirty file page; the **flusher** writes it.
- **Anonymous pages swap**
  Dirty **anonymous** and **shmem** pages are written to swap.
- **Slab uses shrinkers**
  Slab caches are freed by **shrinkers** in the same reclaim round.

What Happens to Each Victim?

How clean and dirty victims are freed.

Same for classic LRU and MGLRU.

A Action by victim type

Pages with no file copy count as dirty until swapped.

	Clean page	Dirty page
Anonymous pages heap, stack, mmap, shmem, tmpfs, etc.	**Drop** Swapped in earlier and unchanged since, so the swap copy is valid. Unmap and free it; no I/O.	**Swap out, then drop** Allocate a swap slot, unmap, and write to swap. Freed after the write completes. Without swap space, these pages cannot be freed this way.
File pages mmap (mapped), read/write (unmapped)	**Drop** Same as the disk copy. Remove it from the page cache and free it; no I/O. It can be read from the file again later.	**Write back, then drop** Reclaim flag is set; the **flusher** writes it. A later round frees it after write-back completes. The page stays in memory until the write finishes.

B Other Cases in a Reclaim Round

- **Slab caches** — freed by shrinkers in the same round
- **Page under writeback** — skipped for now

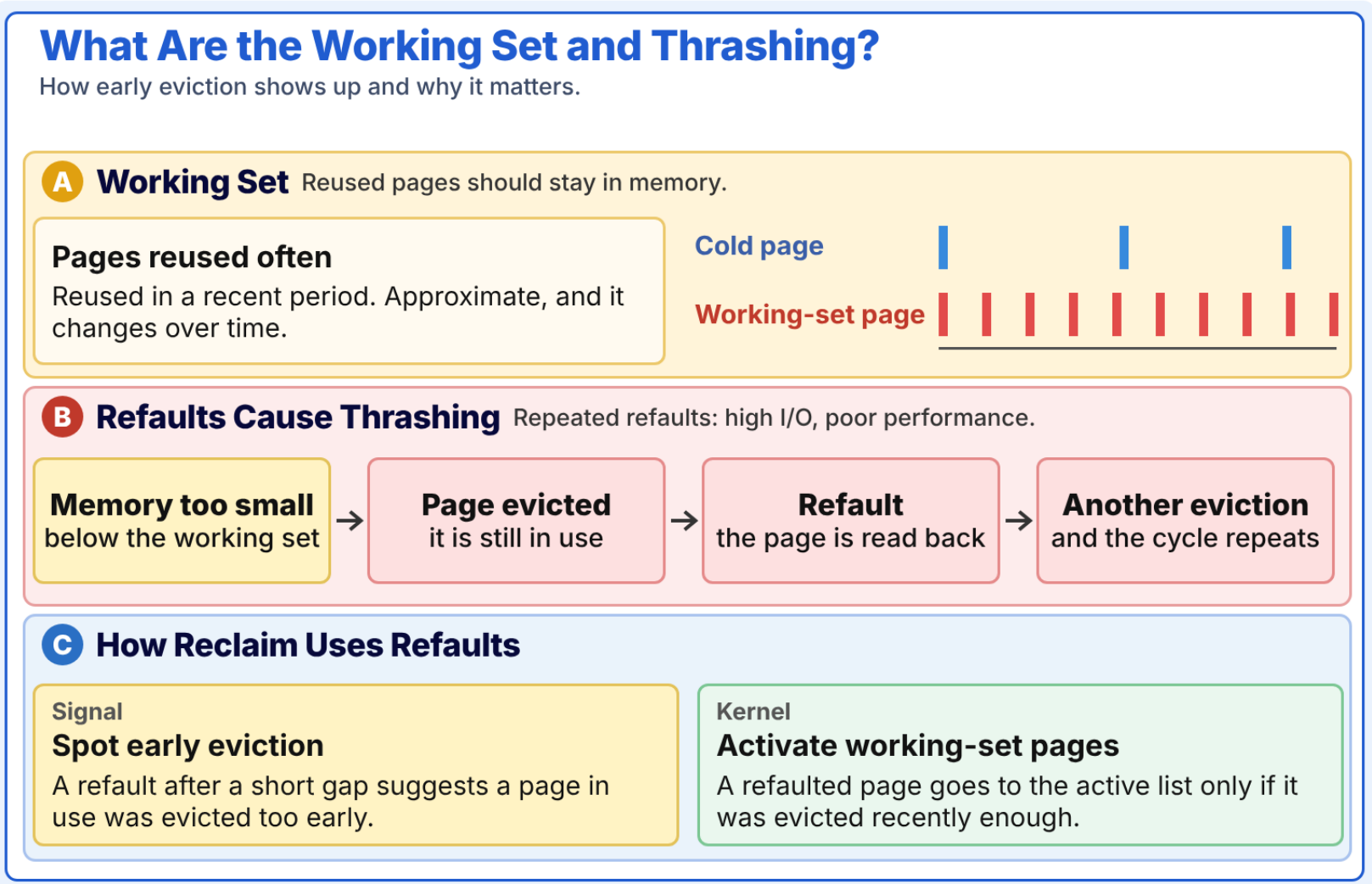
Key Takeaway

A clean victim is dropped; a dirty file page is written by the flusher, a dirty anonymous or shmem page goes to swap, and slab caches are freed by shrinkers in the same round.

What Are the Working Set and Thrashing?

Key Points

- **Working set**
  The **working set** is the pages a program keeps reusing.
- **Refault**
  A **refault** reads back an evicted page; a short gap means it was evicted too early.
- **Thrashing**
  If memory is smaller than the working set, repeated refaults cause **thrashing**.
- **Kernel protection**
  Linux moves a refaulted page to the **active list** only if it was evicted recently.



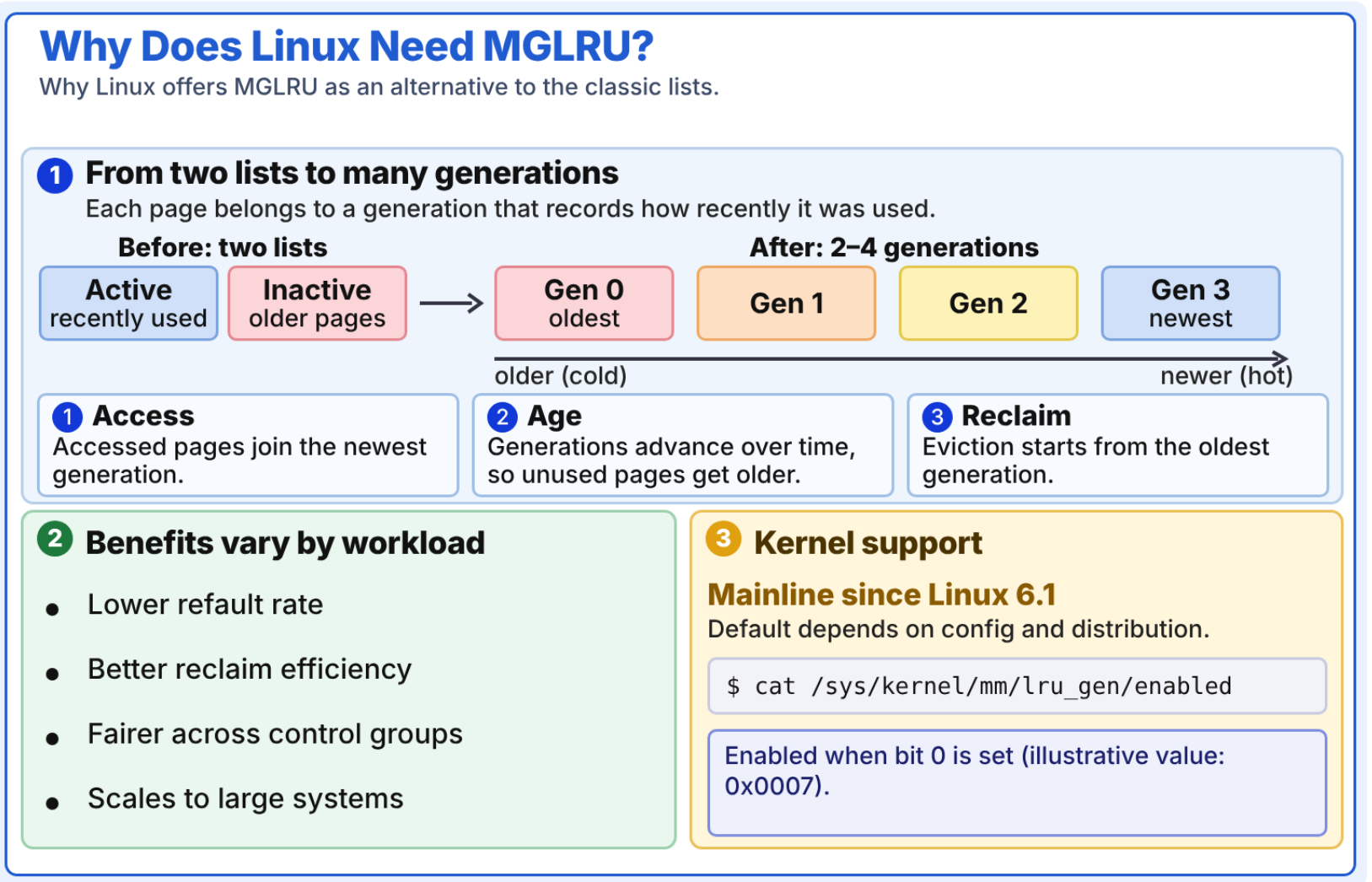
Key Takeaway

A refault can show that reclaim evicted a page too early; the kernel activates pages likely in the working set, but repeated refaults mean thrashing.

Why Does Linux Need MGLRU?

Key Points

- **Short answer**
  MGLRU (Multi-Generational LRU) is an **alternative** reclaim implementation.
- **Why two lists fall short**
  The active and inactive lists give only a **coarse view** of page age.
- **How MGLRU differs**
  Each page sits in a **generation** that matches when it was last accessed.
- **Benefit: better choices**
  Reclaim takes the **oldest generation** first, so refaults and scan cost fall.
- **Availability**
  Mainline since Linux 6.1; defaults vary.



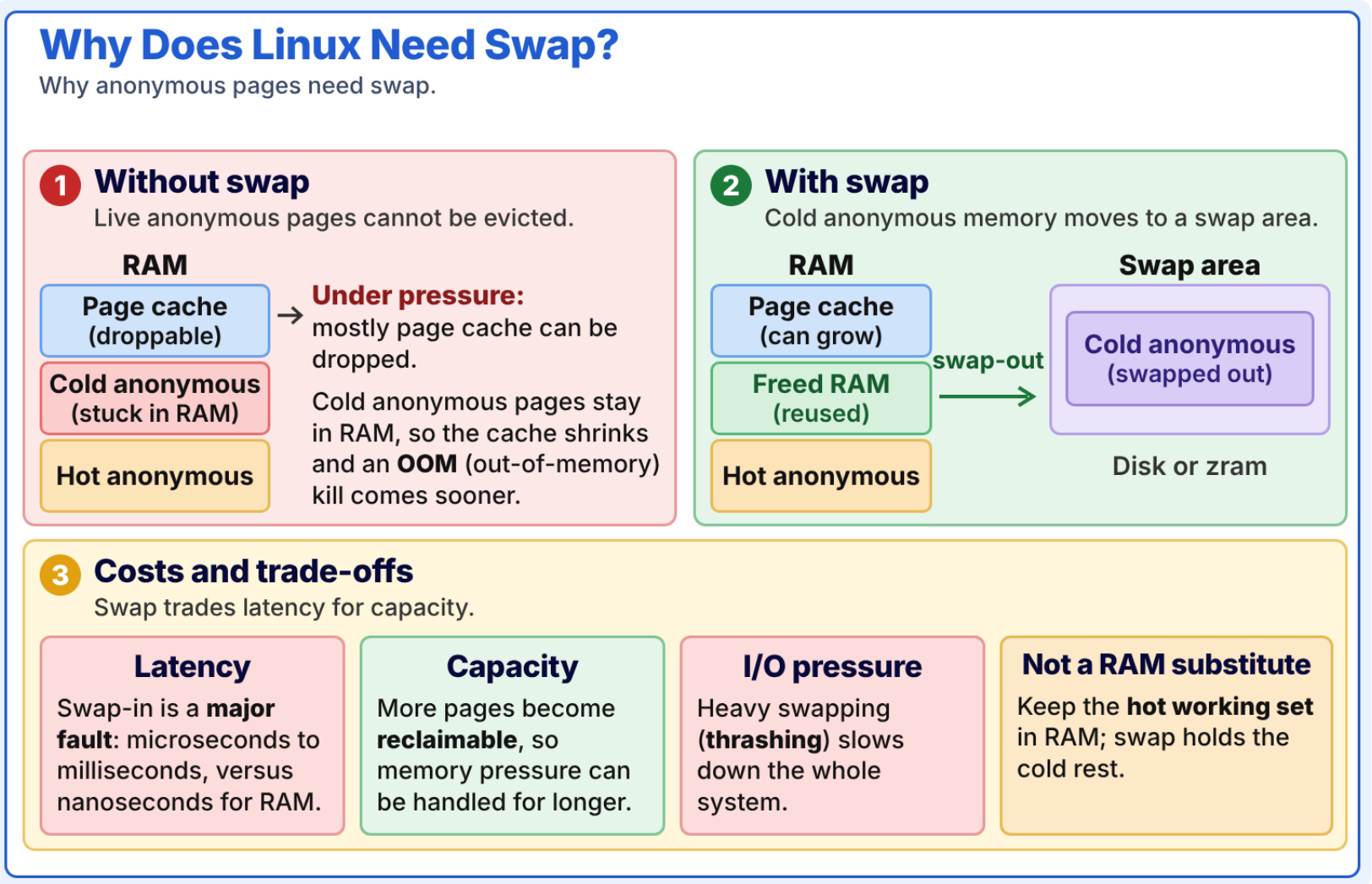
Key Takeaway

MGLRU is an alternative to the classic active/inactive lists: generations rank page age more finely, so reclaiming the oldest first causes fewer refaults.

Why Does Linux Need Swap?

Key Points

- **Swap holds live pages**
  Live anonymous pages have no backing file, so **swap** is where reclaim can write them.
- **Swap in reclaim**
  File pages are dropped or written back; live anonymous pages go to the **swap area**.
- **Cold pages leave RAM**
  Cold pages move to swap, so RAM keeps **hot pages** and the **page cache**.
- **Swap is not extra RAM**
  Swap is far slower than RAM, and heavy swapping causes **thrashing**.



Key Takeaway

Swap gives live anonymous pages a place to be written out, so reclaim can free RAM for hot pages and cache at the cost of latency.

Which Swap Backend Fits: Disk, zram, or zswap?

Key Points

- **Pick by trade-offs**
  Choose **disk swap**, **zram**, or **zswap** by capacity, latency, CPU cost, and flash wear.
- **One path, three stores**
  All three receive pages from the same swap-out path but store the data differently.
- **Disk swap is large**
  Disk swap is large and persistent, but each swap I/O waits for storage.
- **Compressed options**
  zram is compressed swap in RAM; **zswap** is a compressed cache before disk swap.
- **Check the devices**
  swapon --show lists the swap devices, sizes, and priorities.

Which Swap Backend Fits: Disk, zram, or zswap?

Which backend stores them.

1 Where do swapped pages go?

Disk swap	zram	zswap
Reclaim: cold anonymous page	Reclaim: cold anonymous page	Reclaim: cold anonymous page
Write to swap device	Compress the page	Compress into RAM pool
SSD or HDD	zram device in RAM	On eviction: to disk swap

2 Backends compared (typical characteristics)

	Disk swap	zram	zswap
Stored in	Disk (SSD or HDD)	RAM (compressed)	RAM pool + disk swap
Disk I/O	Every swap-out, swap-in	None	Only on eviction or miss
Latency	µs to ms	Fast; needs CPU time	Fast on hit, slow on miss
Capacity	Large	Limited by RAM	RAM pool; disk overflow

3 Check the devices in use (illustrative values)

```
$ swapon --show
NAME        TYPE       SIZE  USED  PRIO
/dev/sda3   partition  8G    1.5G    -2
/dev/zram0  partition  2G    512M   100
```

Disk swap and zram can be active together; the **higher PRIO** device is used first.

Key Takeaway

Choose disk swap for capacity, zram to avoid disk I/O, or zswap to cache compressed pages before disk, trading capacity, latency, and CPU.

How Does zram Work as Swap?

Key Points

- **zram is swap in RAM**
  zram is a block device in RAM that Linux uses as swap, with no disk I/O.
- **Pages are compressed**
  Swap-out compresses each page into zram; a page fault decompresses it again.
- **Savings vary by data**
  Pages that compress well save much memory; poorly compressible pages save little.
- **CPU cost and limits**
  Compression uses **CPU time**; **disksize** and **mem_limit** limit capacity.
- **Check the status**
  zramctl shows the stored data size, compressed size, and total memory used.

How Does zram Work as Swap?

How zram stores them in RAM.

1 zram data path

Both directions stay inside RAM.

Swap-out	Cold anonymous page (selected by reclaim)	→	Compress	→ zram device (compressed pages in RAM)
Swap-in	Page fault (needs a swapped page)	→	Decompress	→ Page back in RAM (anonymous page restored)

2 Capacity depends on compression

Illustrative example (not measured).

Original swapped pages

100 MB

After compression (stored in zram)

45 MB **saves 55 MB**

Good compression: more effective capacity for the same amount of RAM.

Poor compression: savings shrink toward zero.

3 Key settings and status

Typical settings; status values are illustrative.

- `disksize` Logical size shown as swap space.
- `mem_limit` Upper limit on RAM zram may use.
- `comp_algorithm` lz4 or zstd.

```
$ zramctl
NAME   DISKSIZE DATA COMPR TOTAL
zram0         2G 512M 128M  160M
# DATA / COMPR = 4.0x
```

Key Takeaway

zram stores compressed swapped pages in RAM, avoiding disk I/O, but the benefit depends on page compressibility and on the available CPU and RAM.

How Does swappiness Bias Reclaim?

Key Points

- **A bias, not a percentage**
  vm.swappiness sets how strongly reclaim prefers anonymous over file pages.
- **Two kinds of targets**
  Live anonymous pages need swap-out, while clean **file pages** can simply be dropped.
- **How the value works**
  Range 0 to 200 (default 60): anonymous weight is swappiness, file weight is 200 minus it.
- **Zero is not always off**
  With classic LRU, swap can still start under low memory; this varies by kernel version.
- **Match it to swap speed**
  Fast **zram** favors higher values; slow disks favor lower.

How Does swappiness Bias Reclaim?

How reclaim chooses page types.

1 Two reclaim targets

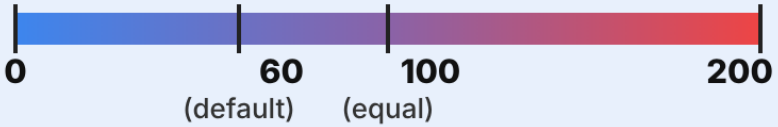
Anonymous pages
heap, stack, anonymous mmap
→ Swapped out to disk or zram

File pages
page cache, executables, mapped files
→ Dropped if clean, re-read from file

2 swappiness: relative bias

vm.swappiness, range 0–200 (kernel 5.8+)

Prefer dropping file pages — Prefer swapping anonymous pages



Not a percentage of memory to swap.

3 How to choose a value

Typical guidance (illustrative values; tune per workload)

Setting	When it fits	Effect on reclaim
Low (example: 10)	Slow swap device (disk, SD card) or flash wear concern	Keeps anonymous pages in RAM; drops file pages first
Default (60)	General-purpose workloads	Balanced; the kernel weighs both by cost
High (100–200)	Fast swap such as zram (compressed RAM)	Swaps rarely used anonymous pages earlier; protects file pages

Key Takeaway

Swappiness biases reclaim between file pages and anonymous pages; it is not a swap percentage, and results also depend on LRU state, refaults, and available swap.

What Is a Shrinker and How Does It Work?

Key Points

- **Shrinker definition**
  A **shrinker** is a callback that kernel caches register to free their objects.
- **Called by reclaim**
  kswapd and **direct reclaim** call shrinkers after scanning the LRU lists.
- **Two callbacks**
  count_objects reports freeable objects; **scan_objects** frees them.
- **Same target**
  Freed slab pages count toward the **same target** as LRU pages.
- **Own scan size**
  The scan size follows **priority** but is separate from the LRU lists.



Key Takeaway

Shrinkers free cached kernel objects in the same reclaim round, after the LRU lists: they share the round's target but have their own priority-based scan size.

When Does Reclaim Make No Progress?

Key Points

- **Few pages freed**
  No progress means many pages are scanned but few are freed.
- **Little to reclaim**
  Locked, **pinned**, and in-use pages cannot be freed.
- **Costly or repeated work**
  Writeback and **swap** take time, and evicted hot pages return at once.
- **Escalation, step by step**
  The **priority number** drops, so the **scan size** grows; then direct reclaim and swap follow.
- **Last resort: a kill**
  Failure can lead to **allocation stalls** and the **out-of-memory (OOM) killer**.

When Does Reclaim Make No Progress?

What happens when reclaim frees too little.

1 Escalation Path Under Pressure

Each step is tried when the previous one frees too little.

Pressure persists → **Priority number drops** (scan size grows) → **Direct reclaim, writeback, swap** → **Still little freed** → **Allocation stalls or retries** → **Possible OOM kill**

Exact retries depend on the allocation context (GFP flags) and system state. Week 10: OOM killer and Low Memory Killer.

2 Why Progress Stalls

Three common reasons.

Small reclaimable pool
Most memory is **pinned or in use**, so few pages can become victims.

Reclaim is costly
Dirty pages need writeback first, and **anonymous** pages need swap space.

Hot pages return
Evicted **hot** pages are used again at once, so the work is wasted.

Result: many pages are scanned, but few are freed.

Key Takeaway

Reclaim makes no progress when the reclaimable pool is too small or too costly to free; persistent failure escalates to allocation stalls and the out-of-memory killer.

Checkpoint: Which Page Goes First?

Key Points

- **Cheap and cold first**
  Pages **cheap to free** and **unlikely to be reused** go first.
- **Check each page**
  Ask its type, whether it is dirty, and whether it is protected.
- **Type sets the cost**
  A **file page** is dropped, a **dirty page** is written back, and **anonymous** needs swap.
- **Protected pages wait**
  Refaulted pages are kept when possible; **mlocked** pages resist reclaim.
- **A tendency, not a rule**
  LRU state, refaults, and swap mean **no fixed order**.

Checkpoint: Which Page Goes First?

Choose which page goes first.

#	Page / object	Reclaimed?	Reason
1	Cold, clean file page (e.g., old readahead data)	Yes, first	Dropped immediately; no I/O needed.
2	Dirty file page (modified, not yet written back)	Later	Must be written back before it can be freed.
3	Cold anonymous page, swap available	Yes, at a cost	Needs swap-out (disk I/O or compression).
4	Cold anonymous page, no swap configured	No	Without swap, anonymous pages cannot be reclaimed.
5	Page that just refaulted (working set)	Yes, last	Recently needed again, so it is kept active; freed only if pressure persists.
6	mlocked or pinned page, in-use kernel memory	No	Unevictable or in use; reclaim cannot free it.

Key Takeaway

Reclaim prefers pages that are cheap to free and unlikely to be reused; the order is a tendency, not a fixed rule. LRU/MGLRU state, refaults, swap, and page state all matter.

PART 3: Memory Management

- **Week 9**

- Memory Pressure and Reclaim Fundamentals
  - When does Linux decide that memory must be reclaimed?
- Page Reclaim Policy, LRU, and Swap
  - Which pages does Linux reclaim, and how does it choose them?
- **Observability, Memory Cgroup, and webOS**
  - How do we observe and control reclaim, and tune it for webOS?

What Is PSI?

Key Points

- **What PSI is**
  PSI is a kernel feature that reports how long tasks **stall** when a resource is short.
- **Time shows the delay**
  Stall time shows the delay tasks feel; usage cannot.
- **Three resources**
  PSI covers **CPU**, **memory**, and **I/O**; idle time is not counted.
- **some**
  some is the share of time at least one task is stalled.
- **full**
  full is the share of time all non-idle tasks are stalled at once.

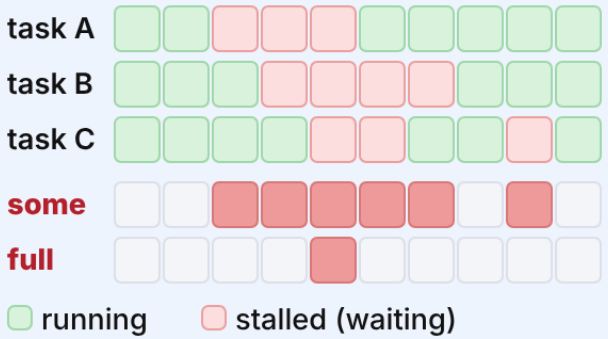
What Is PSI?

Stall time shows the delay that tasks actually feel.

PSI (Pressure Stall Information) is a kernel feature that reports how long tasks **stall** because a resource is short: **CPU**, **memory**, or **I/O**. Idle time is not counted.

1 some vs. full

Illustrative; time runs left to right.



some: any task stalled; **full**: all of them.

2 Two Stall Levels

Share of time, measured over an interval.

some
The share of time **at least one task** is stalled. Other tasks may still run.
Example: 6 of 10 time slots → **60%**

full
The share of time **all non-idle tasks** are stalled at once, so no task makes progress.
Example: 1 of 10 time slots → **10%**

3 What Counts as a Stall

A task stalls when it waits for the resource instead of running.

CPU	Memory	I/O
the task is runnable but waits for a CPU	the task waits for reclaim, compaction, refault, or swap-in	the task waits for block I/O to finish

Key Takeaway

PSI measures how long tasks stall on a resource: some counts time when at least one task stalls, and full counts time when all non-idle tasks stall together.

How Do We Read the PSI Files?

Key Points

- **Same format**
  All three files have a **some** line and a **full** line.
- **Averages**
  avg10, **avg60**, and **avg300** show recent stall percentages.
- **Cumulative total**
  total is the stall time in **µs** since boot.
- **cpu is special**
  For **cpu**, the **full** line is always 0.
- **Exact share**
  Divide the **total** difference by the elapsed time.

How Do We Read the PSI Files?

Three files with one format.

1 Reading Each File

Illustrative values; the format is the same in all three.

```
$ cat /proc/pressure/cpu
some avg10=18.40 avg60=9.20 avg300=2.10 total=4210000
full avg10=0.00 avg60=0.00 avg300=0.00 total=0
```

Tasks waited for a CPU **18.4%** of the last 10 s. **full** stays 0.

```
$ cat /proc/pressure/memory
some avg10=2.10 avg60=0.85 avg300=0.20 total=873400
full avg10=0.30 avg60=0.10 avg300=0.02 total=210000
```

Some task waited for memory **2.1%** of the last 10 s; all together **0.3%**.

```
$ cat /proc/pressure/io
some avg10=5.60 avg60=2.40 avg300=0.50 total=1560000
full avg10=3.10 avg60=1.20 avg300=0.20 total=830000
```

Some task waited for I/O **5.6%** of the last 10 s; all together **3.1%**.

2 Fields

Every line has the same four values.

some
Share of time at least one task is stalled.

full
Share of time all non-idle tasks are stalled. For cpu it is always 0.

avg10 / avg60 / avg300
Moving average of the % of time stalled over about the last 10, 60, and 300 s.

total
Cumulative stall time in µs since boot. It never resets.

3 How to Read Them

Choose the view that matches the question.

Which file?	Quick look: avg	Exact share: total
check cpu, memory, and io to see which resource stalls	avg10 above avg300 means stalls are rising	(total at end − total at start) ÷ elapsed time

Key Takeaway

Read cpu, memory, and io to find the stalled resource: use the averages for a quick look and the total difference for an exact share.

How Do Programs Monitor Memory Pressure With PSI?

Key Points

- **Virtual files**
  The files in **/proc/pressure/** show the **current values**; they are not logs.
- **Two ways to use it**
  A program can **poll** the file, or set a **trigger**.
- **Polling**
  Read the file at **intervals** and compute the stall from the values.
- **Trigger setup**
  Open the file, **write** a condition, and wait with **poll()**.
- **Condition**
  some or **full**, a **stall time**, and a **time window**.

How Do Programs Monitor Memory Pressure With PSI?

Poll the file, or wait for a trigger.

The files in **/proc/pressure/** are **virtual files**, not logs: the kernel builds the **current values** on each read, and nothing is stored on disk.

1 Two Ways to Use PSI in a Program

Each read shows one point in time, so a program reads repeatedly or asks the kernel to notify it.

Polling	Trigger
read the file every interval and check the values	write a condition once; the kernel signals an event

2 Polling

Read periodically and compute.

- **1 Read**
  open and read the file
  at a fixed interval, for example 1 s
- **2 Compute**
  use avg10, or the total difference
  divided by the interval
- **3 Check**
  compare with a limit
  and react when it is exceeded

3 Trigger

Write a condition, then wait for the event.

- **1 Open**
  /proc/pressure/memory
  with O_RDWR | O_NONBLOCK
- **2 Write**
  the condition, for example
  `some 150000 2000000`
- **3 Wait**
  call poll() and sleep until
  the POLLPRI event
- **4 React**
  when the event arrives,
  for example free memory

Format: some or full, then the stall time and the time window in microseconds. **150000 2000000** means **150 ms** of stall within a **2 s** window. Windows range from 500 ms to 10 s.

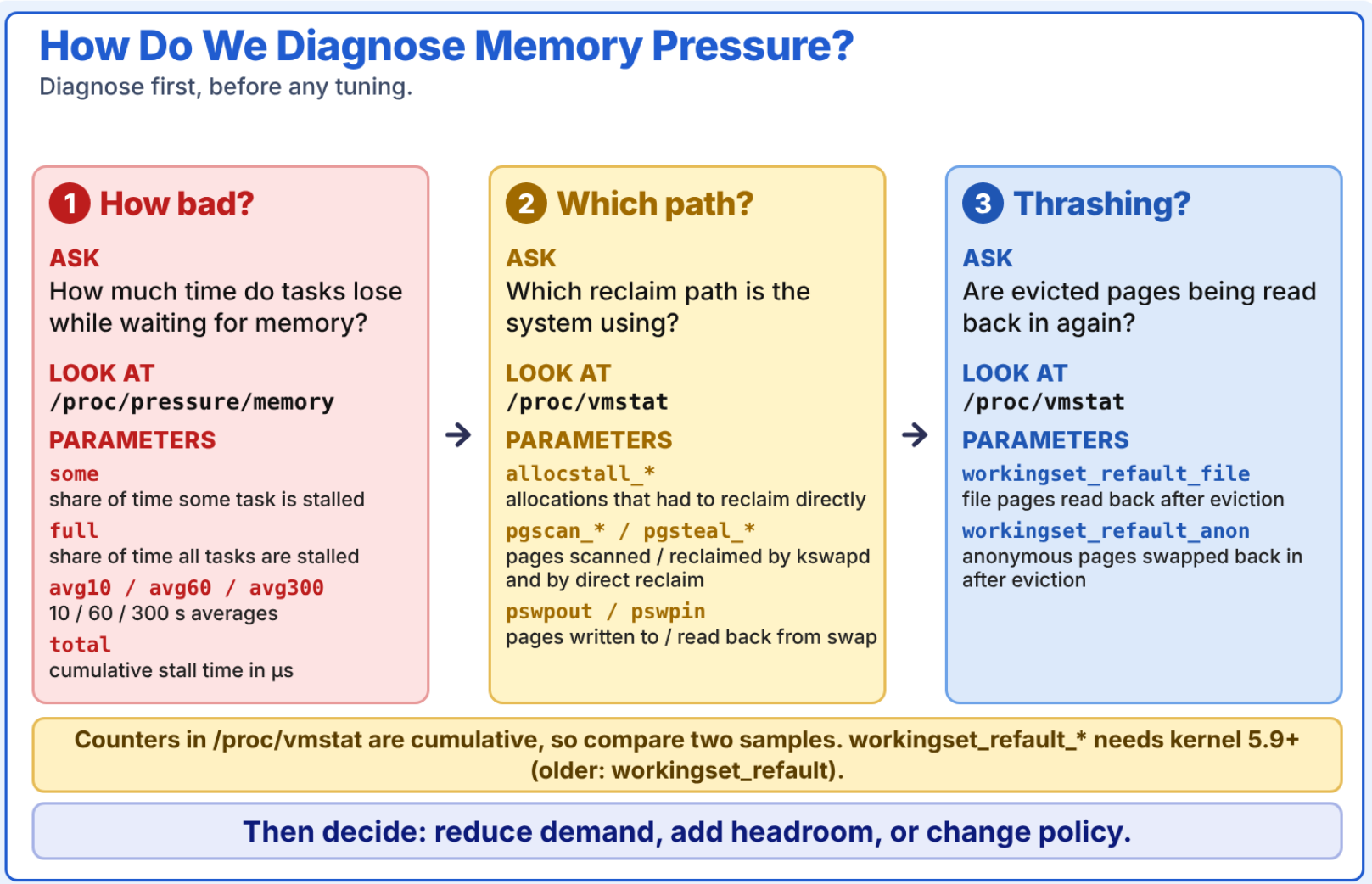
Key Takeaway

PSI files hold only the current values, so a program polls them at intervals or writes a trigger condition and waits for the kernel's event.

How Do We Diagnose Memory Pressure?

Key Points

- **Diagnose before tuning**
  Diagnosis finds out **how bad** the pressure is, **which path** reclaim takes, and whether it is **thrashing**.
- **Ask three questions**
  Ask them in a fixed order: **severity**, then **path**, then **thrashing**.
- **Each has a parameter**
  PSI shows severity, **vmstat counters** show the path, and **refaults** show thrashing.
- **Counters are cumulative**
  Compare **two samples** to see how fast a counter grows.
- **Then choose a fix**
  Reduce demand, add headroom, or change policy; compare **PSI** after.



Key Takeaway

Diagnose before tuning: read PSI for severity, counters for the reclaim path, and refaults for thrashing, then fix and measure again.

Observation: How Bad?

Key Points

- **Same reader, four cases**
  We run one **file reader** in four situations and read the PSI files.
- **Two snapshots**
  Read the **total** at the start and at the end of the run.
- **Subtract and divide**
  The **difference** divided by the run time is the percentage.
- **Not always memory**
  In case 2 the reader slows down, but **memory PSI stays 0**.
- **Compare some and full**
  When another task keeps running, **some** is larger than **full**.

Observation: How Bad?

Real values from one run on a test VM.

Setup: test VM with 2 CPUs, 8 GB RAM, and no swap. One reader loops over a 2 GB file in 1 MB reads for 15 to 25 s per case.

1 Two Snapshots, One Percentage

Case 3 as an example: read the file at the start and at the end of the 25 s run.

```
$ cat /proc/pressure/memory       # at start
some avg10=0.00 avg60=0.00 avg300=0.00 total=149
full avg10=0.00 avg60=0.00 avg300=0.00 total=148
$ cat /proc/pressure/memory       # 25 s later
some avg10=35.44 avg60=12.86 avg300=3.09 total=9576671
full avg10=34.84 avg60=12.68 avg300=3.05 total=9466558
```

1 Subtract the some totals
9,576,671 − 149 = **9,576,522 µs**

2 Divide by the run time
9,576,522 µs ÷ 25,000,000 µs

3 Read it as a percentage
38.3% of the time some task waited

2 Four Scenarios, Same Reader

full and cpu some use the same method with their own totals.

#	Case	Read speed	memory some total start → end	memory some %	memory full %	cpu some %
1	**Healthy**: the file stays in the page cache	8.0 GB/s	0 → 0	0.0	0.0	0.1
2	**CPU contention**: 6 busy-loop tasks	2.3 GB/s	0 → 0	0.0	0.0	99.4
3	**Memory pressure**: another task holds 6 GB	3.2 GB/s	149 → 9,576,671	38.3	37.9	2.3
4	**Pressure + 1 busy-loop task**	2.4 GB/s	9,585,370 → 17,706,155	32.5	26.2	24.7

1 Normal
memory and cpu some are near 0. The file stays in the page cache.

2 Slow, No Memory Pressure
memory some is 0.0%, but **cpu some is 99.4%**. The cause is CPU, not memory.

3 Memory Pressure
some 38.3% and full 37.9%: tasks waited for memory about 4 of every 10 seconds.

4 Memory Pressure + CPU Contention
some 32.5%, full 26.2%, **cpu some 24.7%**. full is lower because another task kept running.

Key Takeaway

Memory PSI separates memory stalls from other slowdowns: it stays 0 under CPU contention and rises when tasks wait for memory.

Observation: Which Path?

Key Points

- **Two situations**
  We run two cases and compare reclaim counters with memory PSI.
- **Background reclaim**
  When only **pgscan_kswapd** grows, kswapd keeps up and PSI stays low.
- **Direct reclaim**
  Rising **pgscan_direct** and **allocstall** mean tasks reclaim for themselves.
- **Tasks wait**
  With direct reclaim, **memory some** was 76.7%, so tasks waited most of the time.
- **Compare two samples**
  Counters are cumulative, so read them twice and **divide the change by the time**.

Observation: Which Path?

Real values from one run on a test VM.

Setup: 2 CPUs and 8 GB RAM. Each value is the change between two samples divided by the run time (4 to 20 s).

1 Two Situations, Per-Second Rates

From /proc/vmstat and /proc/pressure/memory.

#	Situation	memory some %	pgscan_ kswapd	pgscan_ direct	alloc- stall	What it shows
1	**kswapd only** 400 MB/s file reader	0.3	91,376	38	0.1	kswapd keeps up
2	**Direct reclaim** allocates 8.8 GB, swap on	76.7	230,282	405,117	2,484	tasks reclaim for themselves

2 How to Read the Result

Find the path first, then the wait.

kswapd only	**pgscan_kswapd** grows; **pgscan_direct** and **allocstall** stay near 0; **some** is 0.3%.	kswapd keeps up. Its own reclaim time adds a little to **some**.
Direct reclaim	**pgscan_direct** and **allocstall** grow; **some** is 76.7%.	Tasks reclaim for themselves, so they **wait** most of the time.
Other cause	**some** is high, but **pgscan_direct** and **allocstall** stay low.	The wait is not direct reclaim: it is **refault**, **swap-in**, or another wait.

Key Takeaway

pgscan_direct and allocstall show whether tasks reclaim for themselves, and memory some shows how long they wait; a high some without them points to another wait.

Observation: Thrashing or Swap Wait?

Key Points

- **Two more situations**
  We run two more cases: file thrashing and reading back from swap.
- **Thrashing**
  A high **workingset_refault_file** means reclaimed pages are read again.
- **Waiting on refaults**
  During thrashing, **memory some** was 40.5%, so tasks waited often.
- **Swap-out**
  pswpout is a result of pressure, not a wait by itself.
- **Swap-in**
  pswpin means a task waits for a swap read.

Observation: Thrashing or Swap Wait?

Real values from one run on a test VM.

Setup: 2 CPUs and 8 GB RAM. Each value is the change between two samples divided by the run time (4 to 20 s).

1 Two Situations, Per-Second Rates

From /proc/vmstat and /proc/pressure/memory.

#	Situation	memory some %	workingset_refault_file	pswpout	pswpin	What it shows
1	**Thrashing** 6 GB held, 2 GB file re-read	40.5	834,653	0	0	reclaimed pages come back
2	**Swap-in** touches swapped-out pages	11.0	1,593	100,592	106,520	waiting for swap reads

2 How to Read the Result

Find what the task waited for.

Thrashing	**workingset_refault_file** is high; **some** is 40.5%.	**Refault** and some direct reclaim, so tasks wait.
Swap-in	**pswpin** is high; **some** is 11.0%.	Tasks **wait** for swap reads.
Swap-out only	**pswpout** is high, but **pswpin** and **refault** are near 0.	A result of pressure, not a wait by itself.
Other cause	**some** is high, but refault, **pswpin** and direct reclaim stay low.	Check **compact_stall** or other causes.

Key Takeaway

A high refault count shows thrashing and pswpin shows waiting on swap, while pswpout alone only shows that pressure exists.

What Is a Memory Cgroup?

Key Points

- **What a cgroup is**
  A **cgroup** groups processes and accounts for their resources together.
- **Memory cgroup**
  A **memory cgroup** counts and limits the memory of one group.
- **Groups form a tree**
  Groups form a **hierarchy**, and a parent's limit also bounds its children.
- **Per-group state**
  Counters, **LRU lists**, and **PSI** exist per group.
- **Controlled through files**
  In cgroup v2 these are **files** under /sys/fs/cgroup.

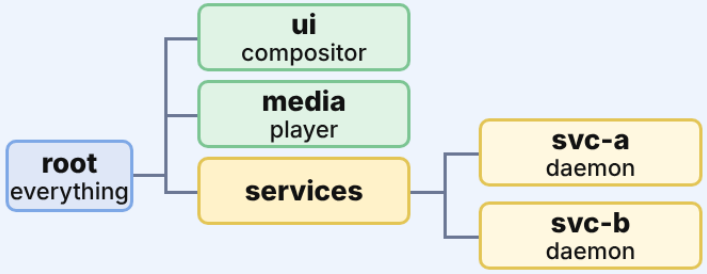
What Is a Memory Cgroup?

How Linux groups processes and their memory.

cgroup (control group) is a kernel feature that groups processes and accounts for and limits their resources as one unit. A **memory cgroup** does this for memory.

1 Groups Form a Tree

Illustrative groups of a device.



A parent group's limit also bounds its children.

2 What Each Group Has

Each `memory.*` name is one file per group.

Usage counters	`memory.current` `memory.stat`
Controls	`memory.min / low / high / max`
LRU lists	its own lists (lruvec), so reclaim can target one group
Status files	`memory.events` `memory.pressure`

3 Using a Memory Cgroup

cgroup v2 example; the memory controller must be enabled in the parent.

1 Create a group
```
mkdir /sys/fs/cgroup/app
```

2 Add a process
```
echo $PID > app/cgroup.procs
```

3 Set a control
```
echo 400M > app/memory.high
```

Key Takeaway

A memory cgroup groups processes and counts and limits their memory as one unit, so reclaim and pressure can be handled per group.

How Do Memory Cgroup Controls Steer Reclaim?

Key Points

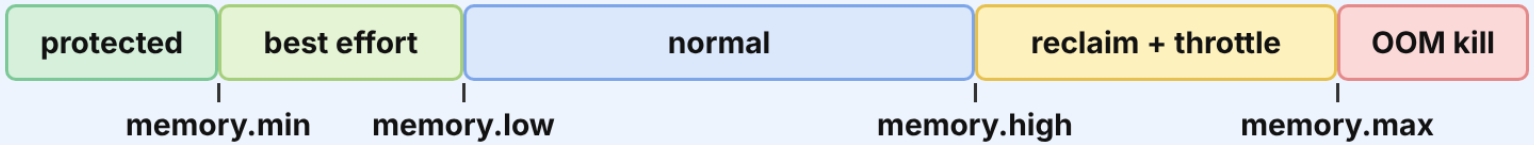
- **Limits steer reclaim**
  Each **memory cgroup** has settings that decide how hard its pages are reclaimed.
- **Per-group accounting**
  Each cgroup keeps its own **usage count** and **LRU lists**.
- **Protect with min and low**
  memory.min is hard protection; **memory.low** is best effort. Both are limited by **parent cgroups**.
- **Limit with high and max**
  Above **memory.high** tasks are **throttled**; **memory.max** is the hard limit.
- **Set in cgroup v2 files**
  Write them under **/sys/fs/cgroup/<group>/**.

How Do Memory Cgroup Controls Steer Reclaim?

Which controls make Linux reclaim one group first.

1 Usage versus the controls

The x-axis is the group's usage (`memory.current`), and these are **not zone watermarks**.



Usual order: min ≤ low ≤ high ≤ max, set per cgroup. OOM = out-of-memory.

2 What Each Control Does

Behavior of one cgroup as its usage grows.

memory.min · hard protection
Reclaim never touches it. If nothing above min is left to reclaim, the OOM killer runs.

memory.low · best-effort protection
Skipped while other cgroups have memory above low; reclaimed only as a last resort.

memory.high · throttle
Above high, reclaim runs on this group and its tasks are throttled. No OOM kill.

memory.max · hard limit
Usage cannot exceed max. If reclaim cannot free enough, a task in this group is OOM-killed.

3 Throttle and Benefits

Throttle means slowing the group's tasks. A task that allocates reclaims this group's pages itself (direct reclaim) and may be put to sleep for a short time.

Isolation
an over-limit group is reclaimed first

Predictability
protected groups keep working sets

Key Takeaway

Memory cgroup controls steer reclaim by protecting groups with min and low, and throttling, reclaiming, or OOM-killing groups at high and max.

What Is the Difference Between memory.high and memory.max?

Key Points

- **High soft, max hard**
  memory.high throttles and reclaims; **memory.max** cannot be exceeded.
- **Both limit one cgroup**
  They apply per **cgroup**; other groups run normally.
- **Above high: throttle**
  The group is **throttled** and **reclaimed**; high never causes an **OOM kill**.
- **At max: OOM possible**
  Direct reclaim runs first; if it fails, an **OOM kill** may follow.
- **Use both together**
  Set **memory.high** below **memory.max** so throttling comes before any **OOM kill**.

What Is the Difference Between memory.high and memory.max?

How the two limits behave differently.

1 Above memory.high

Slow down and reclaim.

- Usage exceeds memory.high
- Tasks in the group are throttled and the group is reclaimed
- Usage falls back below memory.high or stalls keep growing

No OOM kill comes from high itself.

2 At memory.max

Hard limit.

- Allocation hits memory.max
- Direct reclaim runs in the group
- Still no room: cgroup OOM may kill a task in the group

Only tasks in this cgroup are candidates.

3 Use Them Together

Illustrative values; high is set below max.

```
memory.high   400M
memory.max    500M

(high < max)
```

- high: slows the group down early
- max: hard cap as the last defense
- Set high below max so throttling comes first

Key Takeaway

memory.high is a soft limit that throttles and reclaims, while memory.max is the hard limit where failed reclaim can end in a cgroup OOM kill.

Observation: How Is a Memory Cgroup Controlled?

Key Points

- **Set limits on a group**
  We put one process in a group with **memory.high** 400M and **memory.max** 500M.
- **Below high is quiet**
  Under 400M, **memory.events** stays at 0 and **memory.pressure** is near 0.
- **Above high, it slows**
  Usage above **memory.high** raises the **high** count and the group's pressure.
- **At max, a task is killed**
  Hitting **memory.max** with nothing to reclaim raises **max** and **oom_kill**.
- **Only this group**
  System memory stays plentiful; only the **group files** show pressure.

Observation: How Is a Memory Cgroup Controlled?

Illustrative values; one process allocates 700 MB in a group with high 400M and max 500M (no swap).

1 Experiment Setup

cgroup v2 commands.

```
$ mkdir /sys/fs/cgroup/app
$ echo 400M > app/memory.high
$ echo 500M > app/memory.max
$ echo $PID > app/cgroup.procs
$ ./alloc 700M
```

2 What We Observed

Group files while usage grows.

Stage	memory.current	memory.events	memory.pressure
Below high	300M	high 0 · max 0	some ≈ 0
Above high	≈ 450M	high 1,240 · max 0	some ≈ 18
At max	500M	max 18 · oom_kill 1	some ≈ 25

events are counts: high = throttle count, max = limit-hit count, oom_kill = killed tasks.
pressure has the same format as /proc/pressure/memory, for this group only.

System MemAvailable stays about 6 GB; only this group stalls.

3 What It Tells Us

Read the group files as the stages change.

1 Which group?

`memory.events of app`

Only the **app** group is under pressure, even though system memory is plentiful.

2 Soft or hard?

`high vs. max, oom_kill`

high rising means throttling; **max** and **oom_kill** mean the hard limit.

3 How much delay?

`same format as /proc/pressure/memory`

Shows how long tasks in this group **stall** waiting for memory.

Key Takeaway

Even with plenty of system memory, memory.events counts a group's limit events and memory.pressure shows its stall time.

What Can We Tune, and What Does Each Setting Cost?

Key Points

- **Tuning moves cost**
  A setting shifts cost to **free memory**, **CPU**, **I/O**, or **latency**.
- **Three setting groups**
  Watermarks set timing, **swappiness** sets bias, and **cgroup limits** set scope.
- **Each setting has a price**
  Earlier reclaim leaves **less file cache**; more swap adds **I/O or CPU** work.
- **One change at a time**
  Compare **PSI** and **workload latency** before and after each change.
- **Defaults are a start**
  Defaults suit general use; the best values depend on the device and **workload**.

What Can We Tune, and What Does Each Setting Cost?

Which settings can we change, and at what cost?

1 Common settings and what they trade

Recap of earlier slides; effects are workload-dependent and vary by kernel version and environment.

Setting	Typical use	Gain	Cost
`min_free_kbytes`	raise the min watermark (larger reserve)	larger reserve for allocation bursts	less usable memory; earlier reclaim
`watermark_scale_factor`	widen the gap between watermarks	kswapd starts earlier, fewer direct reclaims	less memory for file cache
`swappiness`	bias anonymous vs. file reclaim	protect file cache or anonymous memory	more swap I/O or more cache loss
`zram`	compressed swap in RAM	swap without disk I/O	CPU and RAM used for compression
`memory.high / memory.low`	per-cgroup limit and protection	isolation between groups	throttling of the group

Verify changes with PSI and latency.

Key Takeaway

Each setting trades one cost for another among free memory, CPU, I/O, and latency, so change one at a time and verify with PSI and workload latency.

What Should webOS-Class Devices Do About It?

Key Points

- **One small shared DRAM**
  Apps, UI, **page cache**, and kernel share one small **DRAM**, so memory runs short sooner.
- **Stalls are visible**
  A task waiting in **direct reclaim** shows up as launch delay or jank.
- **Four design directions**
  Keep **headroom**, keep **kswapd** effective, bound each component, and add a cheap swap tier.
- **Reclaim early**
  Ready free pages let a launch skip **direct reclaim**; **kswapd** frees pages early.
- **Bound and watch**
  cgroup limits keep one service from taking others' memory, and **PSI** shows stalls early.

What Should webOS-Class Devices Do About It?

Why small-memory devices stall, and which design choices reduce it.

1 Why Small-Memory Devices Stall Sooner

Illustrative: about 2-4 GB of DRAM shared by all components.

User applications | **webOS framework** | **Compositor and UI** | **Page cache** | **Kernel memory**

When memory runs short, tasks wait for reclaim and users see:

App launch delay | **UI jank (dropped frames)** | **Slow service response**

2 Four Design Directions

Starting points to evaluate on the target device.

Headroom	Background reclaim	Bounded components	Cheap swap tier
Watermarks tuned for launch bursts Reduces: fewer launch-time stalls	**kswapd** gets CPU time before stalls start Reduces: fewer tasks blocked in direct reclaim	**cgroup** limits and protection per service Reduces: one service cannot take others' memory	**zram** if CPU and memory budgets allow it Reduces: more usable memory without disk I/O

Each direction has a cost; validate with **Pressure Stall Information (PSI)** and launch latency.

General design considerations, not a description of any specific webOS implementation. Sizes, values, and settings vary by device and environment.

Key Takeaway

Avoid user-visible stalls on small-memory devices by keeping headroom, bounding components with cgroup limits, and watching PSI early.

Checkpoint: What Do These Signals Mean?

Key Points

- **Each signal's role**
  PSI shows how severe a stall is, and **counters** show which path is active.
- **Three kinds of evidence**
  Read **severity** from PSI, the **reclaim path** from vmstat, the **owner** from cgroups.
- **One signal can mislead**
  A rising **pgscan_direct** alone fits several causes, so it proves none of them.
- **Scope can differ**
  memory.events can show one **cgroup** at its limit while the system looks calm.
- **Confirm, then decide**
  Confirm a reading with a **second counter** before acting.

Checkpoint: What Do These Signals Mean?

Practice reading the signals.

#	Signals (illustrative)	Reading	Check next
1	PSI some low; allocstall flat PSI = Pressure Stall Information	No significant memory stall	Look at CPU and I/O next
2	pgscan_direct up; pgsteal flat	Scanning without progress	Pinned or dirty pages, no swap space
3	pgscan_kswapd up; pgscan_direct flat; PSI some low	kswapd keeps up	Watch the trend
4	workingset_refault_file up; pswpin flat	File cache thrashing	File working set vs. RAM
5	pswpin up; workingset_refault_anon up	Swap-in wait	Swap device and anonymous memory
6	memory.events: max rising	Group hits memory.max	Raise the limit or cut usage
7	memory.events: oom_kill > 0 OOM = out-of-memory	OOM kill inside a cgroup	OOM handling (Week 10)

Key Takeaway

Read signals together: PSI shows severity, counters show the path, and cgroup events show the owner; one value alone rarely proves a cause.

Summary of Week 9

Key Points

- **Pressure is costly**
  Memory pressure is the rising cost of allocation as reclaimable pages shrink.
- **Two ways reclaim starts**
  With no zone above **low**, **kswapd** wakes; below **min**, a waiting task runs **direct reclaim**.
- **Cheap, cold pages first**
  Page **cost and reuse** decide the order, using LRU lists, swap, and writeback.
- **Measure, then tune**
  PSI, counters, and cgroup files show the cause; each setting then trades one cost.
- **Small DRAM, early stalls**
  Small DRAM runs short sooner, so **headroom**, **cgroup limits**, and **PSI** matter.

Summary of Week 9

Key ideas from this week.

1 Fundamentals

When must memory be reclaimed?

- **Pressure** is the rising cost of allocation
- **min / low / high watermarks**: free-page levels that start reclaim
- **kswapd** (background) vs. **direct reclaim** (in the allocating task)
- **Zone-specific pressure**: free pages are counted per zone

2 Policy

Which pages, and why?

- **Cost and reuse** decide which pages are reclaimed first
- **Active / inactive LRU** lists and **MGLRU** (Multi-Generational LRU)
- **Working set** and **refault**: reclaimed pages needed again
- **File vs. anonymous** pages: swap and zram

3 Observe and Impact

What does it cost us?

- **PSI** (Pressure Stall Information) reports pressure as stall time
- **cgroup memory.high** throttles, **memory.max** limits; **memory.events** counts them
- **Thrashing** (repeated reclaim and refault) and tuning trade-offs
- **Small-memory devices** stall sooner; design headroom and limits

Bridge to Week 10

Fewer reclaimable pages and longer stalls bring the system closer to out-of-memory (OOM) conditions. **Less reclaimable memory means an earlier OOM.**

Key Takeaway

Memory pressure is the rising cost of allocation; Linux reclaims low-cost, low-reuse pages first, and PSI and counters measure it before OOM handling.

End of Week 9

Sungyong Park, Ph.D. | **parksy@sogang.ac.kr**

